GMAT CSALT Tutorials and Installation Guide

Peter Edelman, Robert Pritchett, and Peter Candell

Contents

Installing CSALT on Windows 3
CSALT Tutorial: Introduction and Setup 13
Brachistochrone Problem 21
Brachistochrone Problem with Optimal Control Function 30
Conway Low Thrust Spiral 39
Orbit Raising Using Multiple Phases and an OptimalControlFunction 43
DRO to L2 in the CR3BP 52
Hohmann Transfer with GMAT Utilities 58

Installing CSALT on Windows

Contents

MinGW 3
Microsoft Visual Studio 4
CMake 4
SNOPT 7.5 5
Building SNOPT in CMake and Visual Studio 5
GMAT 3rd Party Dependencies 8
Building CSALT 10
Additional Edits to GMAT Base Source Code 8
Compiling CSALT in Visual Studio 11

Download GMAT source code from the source forge mirror

https://sourceforge.net/projects/gmat/files/GMAT/GMAT-R2020a/GMAT-src_and_data-R2020a.zip/download

CSALT requires SNOPT 7.5, a commercially available nonlinear optimizer that is not included with the GMAT package. You will need to acquire SNOPT 7.5 before being able to install CSALT.

To compile SNOPT and CSALT you will need the freely available MinGW Fortran compilers, CMake, and Visual Studio 2022.

MinGW

MinGW provides a free Fortran compiler and runtime library for Windows. This is required because CSALT relies on the SNOPT nonlinear program (NLP) solver package, which is written in Fortran. MinGW can be acquired without administrator privileges by downloading the repository.txt file at this location:

https://sourceforge.net/projects/mingw-w64/files/Toolchains%20targetting%20Win32/Personal%20Builds/mingw-builds/installer/

which provides download URLs for each version of MinGW. CSALT is known to work with MinGW version 7.2.0, build revision 1, 64 bit, posix threads, seh exception handling x86_647.2.0-posix-seh-rt-v5-rev1.2 Download and unzip the archive into a destination with no spaces in the file path.

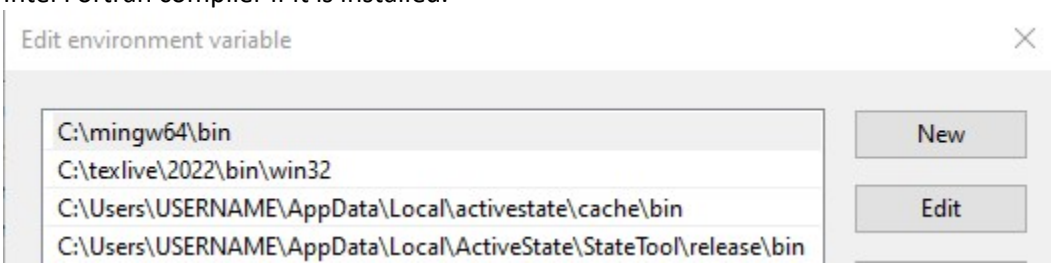
The user should install MinGW to their local machine into a destination with no spaces in the file path and record its location for future use.

After installing MinGW, the user's Path environment variable must be edited to include the bin directory of MinGW so that the CSALT executable can find the Fortran runtime library during execution. To edit

the Path, press the Windows button, then type “Edit environment variables for your account”. (Autocomplete may take over at some point.) In the upper block of the new window, labeled “User variables for <your account name>”, click on “Path”, then click on the “Edit...” button. In the popup window, click “New”. In the new line, type the full path to the MinGW bin directory. An example would be

C:\mingw-w64\x86_64-7.2.0-posix-seh-rt_v5-rev1\mingw64\bin

It is recommended to move the MinGW directory to the top of the list of paths to avoid issues with the Intel Fortran compiler if it is installed.



Microsoft Visual Studio

Microsoft Visual Studio is used to compile and link CSALT. CSALT is known to work with Microsoft Visual Studio 2022 Community Edition, which may be downloaded from:

https://visualstudio.microsoft.com/thank-you-downloading-visual-studio/?sku=Community&channel=Release&version=VS2022&source=VSFeaturesPage&passive=true&tailored=cplus&cid=2031#cplusplus

During the Visual Studio installation process, the user must choose to install the “Desktop development with C++” workload. Executing the Visual Studio installer may require administrator privileges.

CMake

CSALT has a CMake-based build system. CSALT is known to be compatible with CMake 3.23.1, which may be downloaded from:

https://github.com/Kitware/CMake/releases/download/v3.23.1/cmake-3.23.1-windows-x86_64.zip

To avoid the need for administrator privileges, archive and extract the zip file into a destination path with no spaces. Note the location for future use. You may also navigate to the bin directory and right-click the cmake-gui.exe file and select “Pin to Start” for easy access later. CMake will be used to build SNOPT and CSALT.

SNOPT 7.5

CSALT depends on the commercial NLP solver package SNOPT to perform gradient-based optimization. For more information on obtaining SNOPT, see:

http://www.sbsi-sol-optimize.com/asp/sol_product_snopt.htm

The sections below will describe the process for building SNOPT 7.5. When you have acquired SNOPT, place the SNOPT folder it in the GMAT\depends folder of the GMAT directory.

The SNOPT build process described here uses Visual Studio 2022 Community Edition, MinGW x86_647.2.0-posix-seh-rt-v5-rev1, GFortran (provided by MinGW), and CMake 3.23.1.

There are two changes which may need to be made to the SNOPT 7.5 source code. In snopt7\src\sn87sopt.f approximately line 2709, comment (Fortran uses the ! character for comments) out the line that reads:

```
primalInf = primalInf/max(xNorm , one)
```

which can cause SNOPT to incorrectly mark certain solutions as feasible. Additionally, line 235 of sn70nobj.f,

```
call dload ( nF, zero, Elem, 1 )
```

has been shown to cause an error when attempting to build SNOPT in Windows using the MinGW console as opposed to using the CMake and Visual Studio method below. The error can be avoided by replacing the line with:

```
call iload ( nF, 0, Elem, 1 )
```

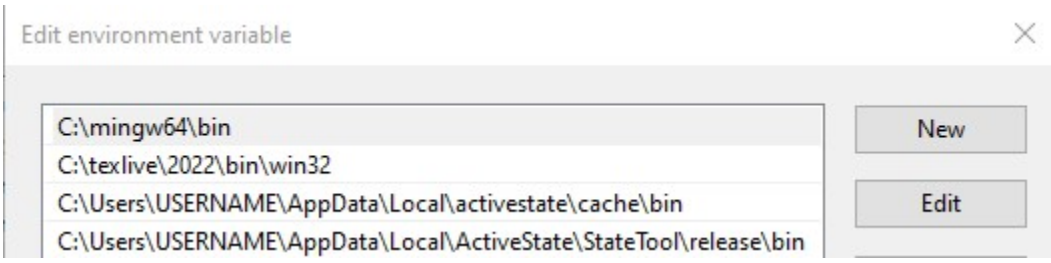
SNOPT 7.5 has two known issues with the CMake file, unlike other SNOPT versions. The first issue is redundant source code that when compiled, will cause repeated symbol errors when linking to the library being built. The second issue is one of the source files calls subroutines that do not exist anywhere in the SNOPT codebase. The way to fix these two issues is to comment them out in the correct CMakeLists.txt file. This CMakeLists.txt file is in the snopt7\src directory of the snopt7 folder. Open the CMakeLists.txt file and comment out the following source code on lines 28, 29, and 53 respectively:

```
sn27lu77.f
sn27lu90.f
snopth.f
```

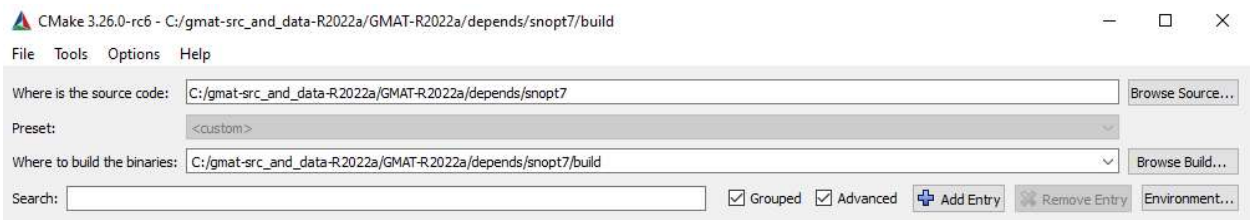
Save this file and you can now make the Visual Studio project files with CMake.

Building SNOPT in CMake and Visual Studio

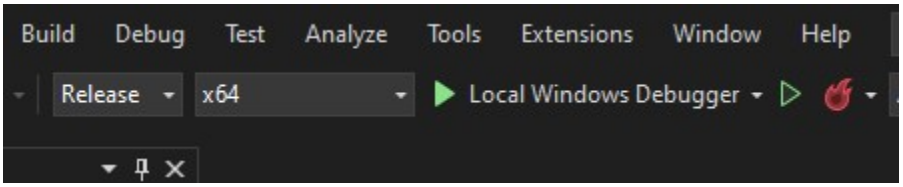
1. Ensure the Windows Path environment variable contains the path to the MinGW bin directory such as C:\mingw\bin. It is recommended that this be placed at or near the top of the list of directories in path (refer to the MinGW section for additional detail).



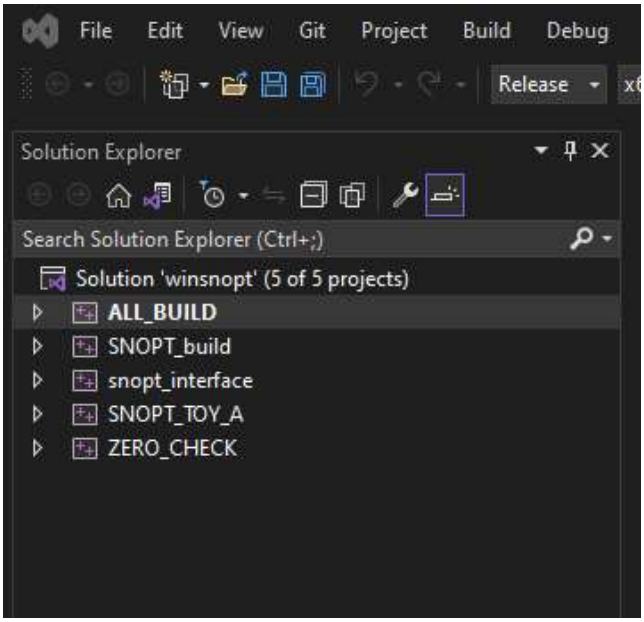
2. Open the CMake GUI and set "Where is the source code" to top-levelSNOPTdirectory. Set "Where to build the binaries" to snopt7\build.
3. Click CMake configure and make sure you are using Visual Studio 2022 toolchain and x64 which should be the CMake default. The MINGW GFORTRAN CMake variable should be the GFortran executable in your MinGW\bin directory. If the Intel Fortran compiler, or something other than GFortran is shown here, the build will likely fail. Windows will often try to default to the Intel Fortran Compiler if installed so ensure MinGW\bin is the first directory in Path and consider removing the Intel Fortran compiler directories from Path.



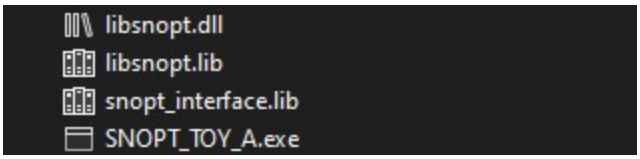
4. Click Generate.
5. Click "Open Project," which should open Visual Studio 2022.
6. Make sure build mode is set to Release.



7. Build the projects in the following order:
    a. SNOPT build
    b. snopt interface
    c. ALL BUILD



8. A successful build will result in a message that says, in part, "0 failed, 0 skipped" in the Visual Studio output window. The snopt7\build\Release directory should now look like this:



Building SNOPT 7.5 produces three libraries required for CSALT:

libsnopt.dll
libsnopt.lib
snopt_interface.lib

libsnopt.dll (or .so on Linux and .dylib on Mac) is the dynamic-link library that contains the SNOPT program. The snopt_interface.lib is the C++ interface that CSALT uses to communicate with Fortran SNOPT library.

GMAT's, and subsequently, CSALT's, current CMakeLists.txt does not recognize the SNOPT libraries with these names. The names of the SNOPT libraries GMAT and CSALT do recognize are snopt7.dll and snopt7_cpp.lib. We can rectify this problem by doing the following:

Change lines 399 and 400 of the GMAT CMakeLists.txt in the main GMAT folder from:

```
FIND_LIBRARY(SNOPT_CPPLIBRARY snopt7_cpp PATHS ${SNOPT_LIB_DIR} NO_DEFAULT_PATH)
FIND_LIBRARY(SNOPT_LIBRARY snopt7 PATHS ${SNOPT_LIB_DIR} NO_DEFAULT_PATH)
```

to:

```
FIND_LIBRARY(SNOPT_CPPLIBRARY snopt_interface PATHS ${SNOPT_LIB_DIR} NO_DEFAULT_PATH)
FIND_LIBRARY(SNOPT_LIBRARY libsnopt PATHS ${SNOPT_LIB_DIR} NO_DEFAULT_PATH)
```

Once these changes are made, place the SNOPT libraries in the folder GMAT\depends\snopt7\lib; you may have to create this folder manually.

GMAT 3rd Party Dependencies

There are a several free 3rd party dependencies GMAT requires. The only dependencies CSALT needs is the C++ Boost library, however the GMAT CMake files will check for the other dependencies. This means you need to have the other dependencies installed so that the CMake files will configure without any issues.

Inside the GMAT\depends folder is a Python script that should automatically download and install all dependencies named configure.py. Before running this script, open the file for editing and ensure the variables vs_version and vs_major_version is set to the correct numbers, corresponding to your Visual Studio version. There is also a proxy flag and variable you can set if you need to get through a proxy wall. ***These instructions assume the user knows how to bypass their proxy wall and is not meant to cover all proxy settings. If your proxy cannot be bypassed with the method in the script, contact your IT department for help.***

Run the configure.py script to download and install the dependencies. ***This will take quite a long time*** due to the total size of the packages. When everything is installed, the script should output, "*** Done configuring GMAT dependencies, please see status messages and logs for details ***"

Required Edits to GMAT Base Source Code and CMake Files

Edits to GMAT Base Source Code

For the sixth tutorial to work correctly, we must add an additional routine to the Ephemeris Smoother source code in the GMAT Base source code called CreateSmoothedEphemForCelestialBody() before compiling CSALT. First, open the header file EphemSmoother.hpp in the GMAT\src\base\solarsys directory for editing. Add the following prototype to the public methods in the EphemSmoother class:

```
virtual void    CreateSmoothedEphemForCelestialBody(CelestialBody *refBody,
                                                    CelestialBody *target,
                                                    CoordinateSystem *refFrame,
                                                    Real theTime0, Real theTimef,
                                                    Integer nRegionsPerRev = 1);
```

Then, add the function itself to the EphemSmoother.cpp source code, also in the GMAT\src\base\solarsys directory:

```
void EphemSmoother::CreateSmoothedEphemForCelestialBody(CelestialBody* refBody,
                                                        CelestialBody* target,
                                                        CoordinateSystem* refFrame,
                                                        Real theTime0, Real theTimef,
                                                        Integer nRegionsPerRev)
{
    std::string refName(refBody->GetName());
    std::string targetName(target->GetName());

    // Implementation currently only supports smoothed
    //  ephemeris with Earth as the central body
    if (refName.compare("Earth") == 0) {
        CelestialBody* targetBody = target;

        // Get state at theTime0
        Rvector6 stateWRTOldOrigin; // zeros
        Rvector6 stateAtT0 = CoordinateTranslation::TranslateOrigin(theTime0,
            targetBody, stateWRTOldOrigin,
            refBody, refFrame);

        // Get gravitational parameter
        Real refGravParam = refBody->GetGravitationalConstant();
        Real targGravParam = targetBody->GetGravitationalConstant();

        // Compute semi-major axis
        Real r = GmatMathUtil::Sqrt(stateAtT0(0) * stateAtT0(0) +
            stateAtT0(1) * stateAtT0(1) +
            stateAtT0(2) * stateAtT0(2));
        Real vsqr = stateAtT0(3) * stateAtT0(3) + stateAtT0(4) * stateAtT0(4) +
            stateAtT0(5) * stateAtT0(5);
        Real a = 1.0 / (2.0 / r - vsqr / (refGravParam + targGravParam));

        // Compute orbital period
        Real period = GmatMathConstants::TWO_PI * GmatMathUtil::Sqrt(a * a * a
            / (refGravParam + targGravParam));

        // Compute step size
        Real stepSize = period / (nRegionsPerRev * 86400.0);

        // Call CreateSmoothedEphemForCelestialBody
        CreateSmoothedEphem(refBody, target, refFrame,
            theTime0, theTimef, stepSize,
            0.0, nRegionsPerRev, "h");
    }
    else
    {
        throw SolarSystemException("ERROR in \
            EphemSmoother::CreateSmoothedEphemForCelestialBody.\
            Currently only supports Earth as refBody input.\n");
    }
```

Edits to CMake Files for CSALT Tester Program

Since the sixth tutorial uses both NAIF/JPL's cspice routines and the GMAT Base library, we need to tell CMake to include them in the associated CMakeLists.txt file. Open this CMakeLists.txt file for editing located in GMAT\src\csaltTester. After line ~138, insert the following:

```
TARGET_INCLUDE_DIRECTORIES(${TargetName} PUBLIC ${BASE_DIRS})
TARGET_INCLUDE_DIRECTORIES(${TargetName} PUBLIC ${CSPICE_INCLUDE_DIR})
```

Then, after ~line 150, add the following:

```
TARGET_LINK_LIBRARIES(${TargetName} PRIVATE GmatBase)
```

A visualization of these parts of the CMakeLists.txt is shown circled below:

```
134     # Add necessary include directories
135     TARGET_INCLUDE_DIRECTORIES(${TargetName} PUBLIC ${CSALT_DIRS})
136     TARGET_INCLUDE_DIRECTORIES(${TargetName} PUBLIC ${CSALT_TEST_DIRS})
137     TARGET_INCLUDE_DIRECTORIES(${TargetName} PUBLIC ${SNOPT_INCLUDE_DIR})
138     TARGET_INCLUDE_DIRECTORIES(${TargetName} PUBLIC ${Boost_INCLUDE_DIR})
139     TARGET_INCLUDE_DIRECTORIES(${TargetName} PUBLIC ${BASE_DIRS})
140     TARGET_INCLUDE_DIRECTORIES(${TargetName} PUBLIC ${CSPICE_INCLUDE_DIR})
141
142     if(PLUGIN_PROPRIETARY_CSALT_C_WRAPPER)
143         TARGET_INCLUDE_DIRECTORIES(${TargetName} PUBLIC ${CSALT_C_WRAPPER_DIRS})
144     endif()
145
146     # Link needed libraries
147     TARGET_LINK_LIBRARIES(${TargetName} PRIVATE ${SNOPT_CPPLIBRARY})
148     TARGET_LINK_LIBRARIES(${TargetName} PRIVATE ${SNOPT_LIBRARY})
149     TARGET_LINK_LIBRARIES(${TargetName} PRIVATE CSALT)
150     TARGET_LINK_LIBRARIES(${TargetName} PRIVATE GmatUtil)
151     TARGET_LINK_LIBRARIES(${TargetName} PRIVATE GmatBase)
```

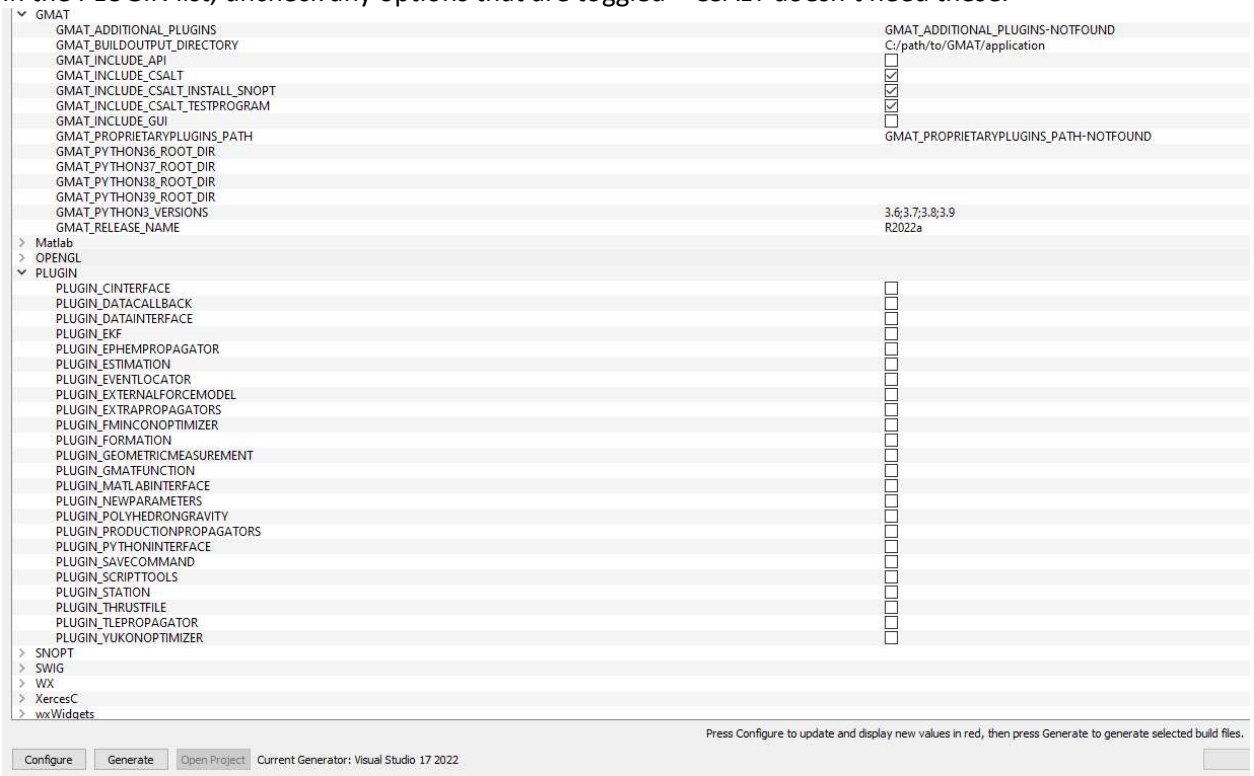
Building CSALT

Setting CMake Options

The CMake program is used to set additional options for both GMAT and CSALT. There is only one other GMAT library required for CSALT, the GmatUtil library. This other library will automatically be compiled along with CSALT.

1. Open the CMake GUI
2. Make sure that the "Advanced" and "Grouped" check boxes are checked. This makes it easier to see all options.
3. In the "Where is the source code:" line, put the path to the GMAT main folder.
4. In the "Where to build the binaries:" line, put a path to a build folder inside the GMAT folder
5. Click "Configure", yes to making a new build directory, have Visual Studio 17 2022 as the generator, and select "Use default native compilers". Click Finish.
6. Upon Configuring, you should have no errors and a "Configuring done" message. There will be a list of other CMake options you can now toggle.
7. In the GMAT list, there is a toggle called "GMAT_INCLUDE_CSALT". Check the box.
8. Uncheck the other boxes in the GMAT list

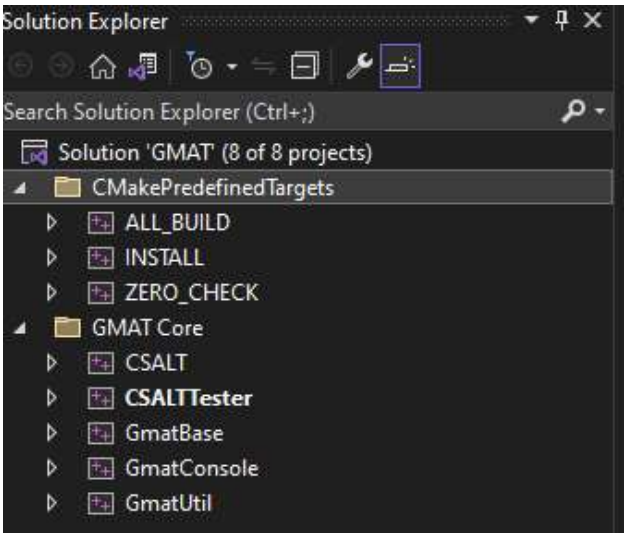
9. In the PLUGIN list, uncheck any options that are toggled – CSALT doesn't need these.



10. Click "Configure".
11. Now CMake should have found SNOPT. If CMake cannot find SNOPT make sure to have done what was outlined in the previous section.
12. In the GMAT list, now there's an option for GMAT_INCLUDE_CSALT_TESTPROGRAM to check. We will make sure CSALT is working with this tester.
13. Click "Configure". Now there should be no CMake menus highlighted in red.
14. Click "Generate" to make the Visual Studio project files
15. Click "Open Project" to run Visual Studio.

Compiling CSALT in Visual Studio

In Visual Studio, the Search Solution Explorer should look like the following after CMake generation:



These are the steps to compile CSALT and run the tester:

1. Ensure that the compile type is set to "Release", not "Debug". There should be a pull-down menu in Visual Studio to select this option.
2. Right-click on the CSALT project and then click Set as Startup Project.
3. Right-click on the CSALT project again and click build. This will build both the GmatUtil and CSALT libraries.
4. When finished the prompt should read "Build: 3 succeeded"
5. Right-click on the CSALTTester project and click build.
6. There should read "Build: 1 succeeded" on the prompt when finished.
7. Right-click on the CSALTTester project again and click Set as Startup Project.
8. Type Ctrl + F5 to run the program. Run any of the test cases and verify CSALT is working correctly.
    a. If you get an error saying that libsnopt.dll (or .so on Linux and .dylib on Mac) cannot be found, a workaround is to add the directory where libsnopt.dll is located to your Windows Path environment variable. We can do this by:
        i. Control Panel -> System -> Advanced system settings -> Environment Variables…
        ii. In the System variables list (second one) click on the Path Variable and then Edit…
        iii. Click New and add the directory where libsnopt.dll is located (GMAT\depends\snopt7\lib)

Here are some notes on where libraries and executables are stored, and the CSALT output files:

- The static export libraries for GmatUtil (GmatUtil.lib) and CSALT (CSALT.lib) are in \build\src\gmatutil\Release and \build\src\csalt\Release respectively
- The dynamic libraries for GmatUtil (libGmatUtil.dll) and CSALT (libCSALT.dll) are in GMAT\application\bin
- The CSALTTester executable is located at GMAT\application\bin\CSALTTester.exe
- Upon running a CSALT problem, two output files are produced:
    - *.och files – formatted output of the independent variable, state variables, and controls
    - *Data.txt files – SNOPT screen output

CSALT Tutorial: Introduction and Setup

Contents

Learning Objectives 13
Creating a Visual Studio csaltTutorials Project 14
Compiling Individual Tutorial Problems 15
Creating a CSALT Optimal Control Problem with the Template 17
PointObj – Using the CSALT UserPointFunction Class 18
Additional Methods in the UserPointFunction Class 18
PathObj – Using the CSALT UserPathFunction Class 19
Additional Methods in the UserPathFunction Class 19
Driver – Using the CsaltDriver Helper Class 17
Methods to Set Up Phases with CsaltDriver Class 17
CSALT Output 20

Welcome to the tutorial series on NASA's Collocation Stand-Alone Library and Toolkit (CSALT), which is part of NASA's General Mission Analysis Tool (GMAT). CSALT is a C++ library that uses direct transcription to convert optimal control problems into a large, sparse Nonlinear Programming (NLP) problem. The transcription methods are different forms of collocation, an implicit form of numerical integration. CSALT has three dependencies to run separate of GMAT:

1. C++ Boost library header files
2. GmatUtil library
3. SNOPT v7.5

The C++ Boost library is available for free online and is already part of the GMAT third-party dependencies. The GmatUtil library is automatically compiled when compiling CSALT. SNOPT v7.5 (Sparse Nonlinear OPTimizer) is commercially available software and does not come standard with GMAT or CSALT. This tutorial assumes you have successfully completed the installation of CSALT and the CSALTTester program as outlined in the "Installing CSALT on Windows.docx" MS Word document.

Learning Objectives

In this tutorial we will describe how to create a CSALT project in Visual Studio, describe the main CSALT C++ base classes that must be inherited to use the tool, and five example problems we will walk through. Each tutorial sequentially adds additional CSALT features available to create and solve optimal control problems.

The five CSALT tutorials are provided below with a description of the objective we will work on with each problem.

1. **Brachistochrone** – Introduction in how to set up a CSALT problem with the UserPointFunction and UserPathFunction base classes.
2. **Brachistochrone with OptimalControlFunction** – Similar to the first tutorial, but we will describe how to use the OptimalControlFunction class that may be beneficial for many problems.
3. **Conway Low Thrust Spiral** - Show how mesh refinement is used to improve the accuracy of a solution, and how to use a more complex dynamical model.
4. **Multi-Phase Orbit Raising with OptimalControlFunction** - Show how to set up a multi-phase problem. Similar dynamics to the Conway Low Thrust Spiral but now with time-varying mass. Uses the OptimalControlFunction class to separate terminal and phase linkage constraints.
5. **DRO to L2 Transfer** – Capstone problem to solve a complicated low thrust transfer in the Earth-Moon circular-restricted three-body problem (CR3BP). This is a transfer from a lunar Distant Retrograde Orbit (DRO) to a periodic orbit around the Lagrange point L2.

First, we need to set up a project for these tutorial problems in Visual Studio.

Creating a Visual Studio csaltTutorials Project

The easiest way to create a project for our tutorials is to copy the csaltTester project, with a few edits. Creating the project this way leaves the third-party dependency setup intact. To create the csaltTutorials project do the following (BUILD is the build folder where CSALT and csaltTester were created, nominally located at GMAT\BUILD). Note that some of these steps need to be repeated if you are switching between Release and Debug mode.

1. In BUILD, create a folder called csaltTutorials
2. Navigate to BUILD\src\csaltTester
3. Copy the following files into the BUILD\csaltTutorials folder:
   CSALTTester.vcxproj
   CSALTTester.vcxproj.filters
4. Rename the copied files to:
   CSALTTutorial.vcxproj
   CSALTTutorial.vcxproj.filters
5. Open CSALTTutorial.vcxproj in Visual Studio
6. Let the project load, and then right-click on the project that is (for now) incorrectly named CSALTTester, then click on Rename. Rename to CSALTTutorial. Make sure to be in Release mode, and not Debug mode.
7. In the newly named CSALTTutorial, open the Header Files list and remove all header files except ConsoleMessageReceiver.hpp. Ctrl + Shift + click will help with removing the files
8. Open the Source Files and remove all source files except ConsoleMessageReceiver.cpp
9. In the BUILD\csaltTutorials folder create a \src folder and an \include folder
10. In the \src folder, add the following file included in the samples package located in \application\samples\CSALT\src:
    CsaltDriver.cpp
11. In the \include folder, add these files included in the samples package located in \application\samples\CSALT\include:
    CsaltDriver.hpp
    Driver.hpp

OcfObj1.hpp
OcfObj2.hpp
PathObj.hpp
PointObj.hpp

12. In Visual Studio, right-click on Source Files → Add → Existing Item… and add the CsaltDriver.cpp file located in the \src folder
13. Right-click on Header Files → Add → Existing Item… and add the following files, located in the \include folder:
    CsaltDriver.hpp
    Driver.hpp
    OcfObj1.hpp
    OcfObj2.hpp
    PathObj.hpp
    PointObj.hpp
14. Click on the CSALTTutorial project (top of the tree) then in the menu at the top of Visual Studio, click Project → Properties
15. In the General tab, edit the Output Directory to where you want the compiled executable to be located. E.g. BUILD\csaltTutorials\bin.

    NOTE: If this location is not set to the same Application\bin in a locally built version of GMAT or \bin in a downloaded version of GMAT, the application extension files (.dll on windows, .so on linux, and .dylib on mac for example) will need to be copied over into the chosen \bin folder in order to run CSALT. An error will be thrown when running CSALT that will state the files that need to be copied over.

16. In Linker → Input, open the Additional Dependencies for editing
17. The lines:
    ..\csalt\Release\CSALT.lib
    ..\gmatutil\Release\GmatUtil.lib
    should be edited to where they are correctly located in:
    BUILD\src\csalt\Release\CSALT.lib
    BUILD\src\gmatutil\Release\GmatUtil.lib
18. Ensure the lines linking to the SNOPT libraries are also correct in the linker options:
    GMAT\depends\snopt7\lib\snopt_interface.lib
    GMAT\depends\snopt7\lib\libsnopt.lib
19. Click OK then Apply.
20. Ensure both the BUILD\csaltTutorials\src and BUILD\csaltTutorials\include folders you created above are in the C/C++ → General → Additional Include Directories.
21. Click Apply then OK to exit the Properties Window.

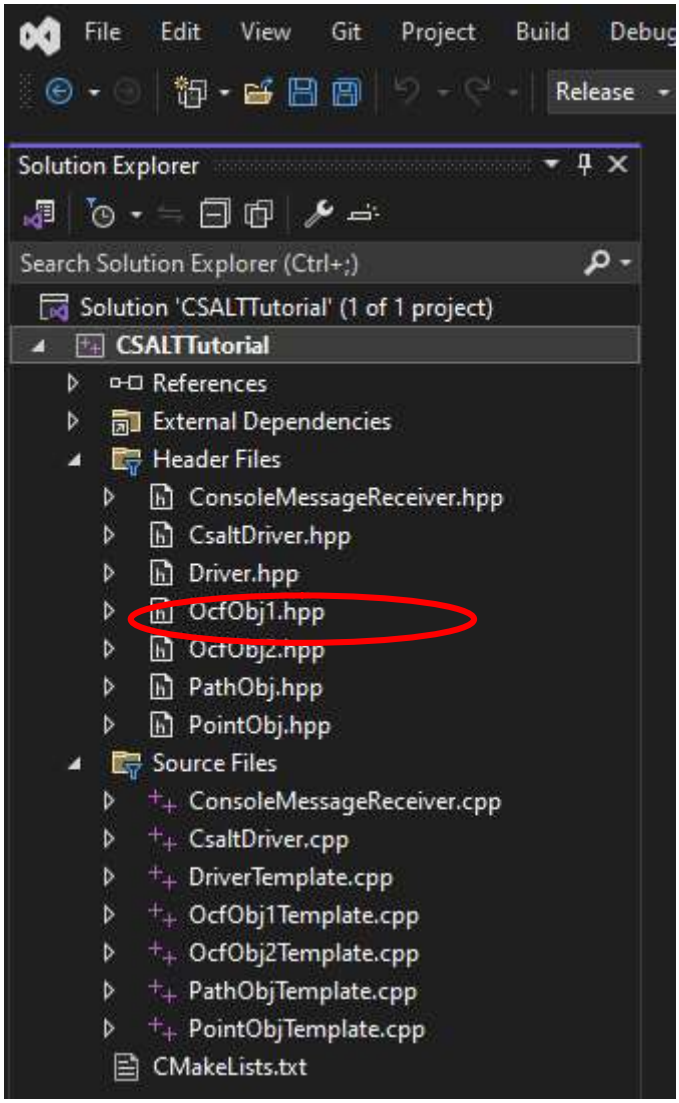
Now the CSALTTutorials project should be ready to be built.

Compiling Individual Tutorial Problems

There are a number of "Template" source files that are used to construct the tutorial problems:
DriverTemplate.cpp
PointObjTemplate.cpp

PathObjTemplate.cpp
OcfObj1.cpp
OcfObj2.cpp

We will be using these files as the foundation to construct each tutorial problem. The DriverTemplate.cpp source file includes the C++ main() function, indicating it is the beginning of a given program. This means that when we create the executable for each tutorial problem, we need to swap out the file that includes this main() function in the Source Files folder in Visual Studio. For example, if we rename DriverTemplate.cpp to DriverBrach.cpp for the first tutorial, then rename DriverTemplate.cpp to DriverBrachOCF.cpp for the second tutorial, only one of these two files can be present in the Source Files folder in Visual Studio when building the executable. If both are included, the compiler will throw an error that more than one main() function is found, meaning the beginning of the program is undefined.



We build each tutorial program by clicking on Build -> Build Solution (or F7). Upon successful compilation, we can run it by clicking Debug -> Start Without Debugging (or Ctrl + F5).

Creating a CSALT Optimal Control Problem with the Templates

The CSALT tutorial comes with templates that we will use to construct the optimal control problems. Also included is a helper class, CsaltDriver.cpp, that assists in setting up the problem phases, dynamics, bounds, and constraints. The helper class also automatically performs memory allocation and interfaces with the CSALT SNOPT optimizer class to solve the optimal control problem. Note that it is not necessary to use the CsaltDriver class to run CSALT but exists as a convenience to the user.

Driver – Using the CsaltDriver Helper Class

The Driver class uses the CsaltDriver base class that is *not* required to run CSALT. The CsaltDriver helper class is included in these tutorials to automatically handle memory management, set up the problem phases, and run the optimization problem with SNOPT.

There are three required methods to define for the Driver class. The first required method is SetPointPathAndProperties(), which sets up the required UserPointFunction and UserPathFunction classes, and any optional OptimalControlFunction classes. This method is also used to set or define available collocation or optimization settings. The second require method is SetupPhases(), which unsurprisingly, sets up each of the phases for a CSALT problem. The third required method is PostProcess() where the user can perform post-processing operations on the optimized trajectory if desired.

Methods to Set Up Phases with CsaltDriver Class

Listed below are the available methods used to set up phases for CSALT using the CsaltDriver class. First a phase object is created with the desired collocation method. The available methods are Hermite-Simpson, Lobatto IIIa (orders 4, 6, and 8) and Radau collocation. An example with Radau collocation is

```
RadauPhase *phase = new RadauPhase();
```

This new phase object point will be used to construct a new phase. The available methods for this phase object are below. Most methods are self-explanatory by their names:

- phase->SetInitialGuessMode(std::string IGMode):
  IGMode can be one of the following:
  - LinearNoControl – Initial guess based on linearly interpolating between minimum/maximum bounds. Constant zero is used for the controls guess.
  - LinearUnityControl – Initial guess based on linearly interpolating between minimum/maximum bounds. Constant one is used for the controls guess
  - LinearCoast – Similar to LinearNoControl
  - OCHFile – Use a formatted *.och file as the input for the initial guess
  - GuessArrays – User fills in initial guess into arrays manually
- phase->SetInitialGuessArrays(Rvector time, Rmatrix state, Rmatrix control):
  User input initial guess for “GuessArrays” initial guess mode
- phase->SetNumStateVars(Integer n)
- phase->SetNumControlVars(integer m)
- phase->SetMeshIntervalFractions(Integer mif)
- phase->SetMeshIntervalNumPoints(Integer minp)
- phase->SetStateLowerBound(Rvector lowerBound)

- phase->SetStateUpperBound(Rvector upperBound)
- phase->SetStateInitialGuess(Rvector st0)
- phase->SetStateFinalGuess(Rvector stf)
- phase->SetTimeLowerBound(Real tLower)
- phase->SetTimeUpperBound(Real tUpper)
- phase->SetTimeInitialGuess(Real t0)
- phase->SetTimeFinalGuess(Real tf)
- phase->SetControlLowerBound(Rvector uLower)
- phase->SetControlUpperBound(Rvector uUpper)

For the SetStateInitialGuess(), SetStateFinalGuess(), SetTimeInitialGuess(), and SetTimeFinalGuess(), put values in between their bounds to be close to what you think they would be. SNOPT will figure out what they should be upon getting a converged solution.

PointObj – Using the CSALT UserPointFunction Class

The PointObj class uses the required CSALT base class, UserPointFunction. UserPointFunction has methods for setting constraints and bounds on the dynamics and their derivatives. These constraints and bounds can be enforced on the terminal (first and last) boundaries, as well as between each phase.

There are two required methods that must be defined for the UserPointFunction class. The first, EvaluateFunctions(), contains the objective function and evaluates any desired constraints and bounds. Optionally, constraints and bounds can be evaluated separately in the OptimalControlFunction class which we will go over in the second tutorial problem. The second required method is EvaluateJacobians() where constraints and bounds are set on the derivatives of the dynamics (this method is often not used and left blank).

Additional Methods in the UserPointFunction Class

There are additional methods that can be used in the UserPointFunctionClass to set up the objective function, the constraints, and the constraint bounds:

- Rvector GetInitialStateVector(Integer n):

Obtains the initial state vector for phase n and stores it in a GMAT Rvector data type.

- Rvector GetFinalStateVector(Integer n):

Obtains the final state vector for phase n and stores it in a GMAT Rvector data type.

- Real GetInitialTime(Integer n):

Obtains the initial time for phase n.

- Real GetFinalTime(Integer n):

Obtains the final time for phase n.

- SetFunctions(FunctionType ftype, Rvector rvals):

Sets different functions based on input ftype and values rvals:

- DYNAMICS – sets values for functions on the dynamics
- COST – sets objective function
- ALGEBRAIC – sets values for general algebraic boundary and phase constraints

- SetFunctionBounds(FunctionType ftype, FunctionBound fbound, Rvector rvals):

Sets different function bounds for everything set with the SetFunctions() method. ftype takes:

- DYNAMICS – sets values for bounds on the functions related to the dynamics
- COST – sets objective function bounds
- ALGEBRAIC – sets values for bounds on general algebraic boundary and phase constraints

fbound takes:

- LOWER – set lower bounds
- UPPER – set upper bounds

PathObj – Using the CSALT UserPathFunction Class

The PathObj class uses the required CSALT base class, UserPathFunction. UserPathFunction contains the dynamics and their derivatives for each phase. Like the UserPointFunction class, there are two required methods that share the same names of EvaluateFunctions() and EvaluateJacobians(). In the UserPathFunction class, EvaluateFunctions() is used to define each phase's dynamics and any desired path constraints. EvaluateJacobians() defines the Jacobians (first-derivatives) for the dynamics and path constraints with respect to time, the states, and the controls.

Additional Methods in the UserPathFunction Class

There are additional methods that can be used in the UserPathFunctionClass to set up the dynamics, Jacobians, and path constraints:

- Rvector GetStateVector():

Obtains the state vector for use in the dynamics, Jacobians, and path constraints and sets it to a Rvector data type

- Rvector GetControlVector():

Obtains the control vector for use in the dynamics, Jacobians, and path constraints and sets it to a Rvector data type

- Real GetTime():

Obtains the current time for use in the dynamics, Jacobians, and path constraints

- SetFunctions(FunctionType ftype, Rvector rvals):

Sets different functions based on input ftype and values rvals:

- DYNAMICS – sets the dynamical state equations
- ALGEBRAIC – sets path constraints

- SetFunctionBounds(FunctionType ftype, FunctionBound fbound, Rvector rvals):

Sets different function bounds for everything set with the SetFunctions() method. ftype takes:

  - ALGEBRAIC – sets values for general algebraic boundary and phase constraints

fbound takes:

  - LOWER – set lower bounds
  - UPPER – set upper bounds
- SetJacobian(FunctionType ftype, JacobianType jtype, Rmatrix mvals):

Sets the values for the Jacobians on the dynamics and path constraints. ftype takes:

  - DYNAMICS – sets values for the Jacobian on the dynamics
  - ALGEBRAIC – sets values for the Jacobian on the path constraints

The JacobianType, jtype, takes:

  - STATE – sets values of the Jacobian with respect to the state
  - CONTROL – sets values of the Jacobian with respect to the control
  - TIME – sets values of the Jacobian with respect to time

CSALT Output

Upon running CSALT, two output files are generated. The first is a *.och file which is a formatted output file of the time, states, and controls at the collocation points. The information is separated by the number of phases in the problem. The second file is a *Data.txt file that has all of the SNOPT output and diagnostics – anything having to do with the NLP is shown in this output file.

Brachistochrone Problem

Contents

Problem Formulation 21
Driver 22
Setting up UserPointFunction 24
Setting up UserPathFunction 25
Main Function 28
Building and Running the Problem 28

Problem Formulation

Find the shape of a frictionless wire that results in the minimum transfer time for a bead sliding between two points on the wire.

$$Min\ J = t_f$$

Subject to:

$$\dot{x} = V \sin u$$

$$\dot{y} = V \cos u$$

$$\dot{V} = g_0 \cos u$$

Initial conditions:

$$t_0 = 0$$

$$x(t_0) = 0$$

$$y(t_0) = 0$$

$$V(t_0) = 0$$

Final conditions:

$$0 \leq t_f \leq 10$$

$$x(t_f) = 1$$

$$-10 \leq y(t_f) \leq 0$$

$$-10 \leq V(t_f) \leq 0$$

Variable bounds:

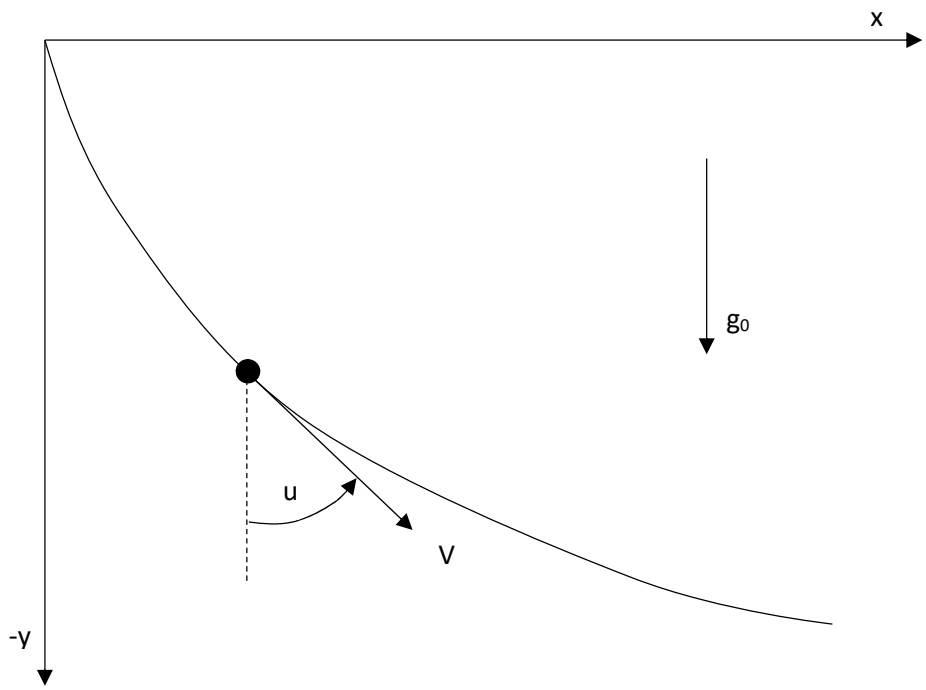
$$-10 \leq x(t) \leq 10$$

$$-10 \leq y(t) \leq 0$$

$$-10 \leq V(t) \leq 0$$

$$-10 \leq u(t) \leq 10$$

With constant g_0 = -32.2. See picture below for visualization.



Driver

First, we fill in the "Driver" class which uses a helper base class CsaltDriver. Note that CsaltDriver is *not* required to create CSALT applications, and one can set up the problem phases and run the optimizer manually if desired. However, the UserPathFunction and UserPointFunction classes *are* required for CSALT.

```
// SetPointPathAndProperties() method is where the Path, Point, and
// OptimalControlFunction classes will be instantiated.
void Driver::SetPointPathAndProperties() {
    pathObject = new PathObj();
    pointObject = new PointObj();

    // Set CSALT and SNOPT-related parameters in this method
    // See CsaltDriver.hpp for some of the available
    maxMeshRefinementCount = 25;
    feasibilityTolerances(0) = 1.0e-6;
    optimizationMode[0] = "Minimize";
}

void Driver::SetupPhases() {
    //Set mesh properties
    RadauPhase* phase1 = new RadauPhase();
    // Initial guess modes: "LinearNoControl", "LinearUnityControl",
    // "LinearCoast", "OCHFile", "GuessArrays"
    std::string initialGuessMode = "LinearNoControl";
    Rvector meshIntervalFractions(3, -1.0, 0.0, 1.0);

    IntegerArray meshIntervalNumPoints = { 5, 5 };

    //Set time properties
    Real timeLowerBound = 0.0;
    Real timeUpperBound = 100.0;
    Real initialGuessTime = 0.0;
    Real finalGuessTime = 0.3;

    //Set state properties
    Integer numStateVars = 3;
    Rvector stateLowerBound(3, -10.0, -10.0, -10.0);
    Rvector stateUpperBound(3, 10.0, 0.0, 0.0);
    Rvector initialGuessState(3, 0.0, 0.0, 0.0);
    Rvector finalGuessState(3, 2.0, -1.0, -1.0);

    //Set control properties
    Integer numControlVars = 1;
    Rvector controlUpperBound(1, 10.0);
    Rvector controlLowerBound(1, -10.0);
```

The Phase object contains the configuration for the selected transcription, the bounds on optimization parameters (e.g. time, state, and control), and the initial guesses for those parameters. In this example, the phase is configured to use Radau orthogonal collocation with two equally spaced mesh intervals, each containing five points (meshIntervalNumPoints). Note that the elements in meshIntervalFractions must span the number space [-1, 1] and be monotonically increasing. Additionally, the number of

elements in meshIntervalNumPoints must be one less than the number of elements in meshIntervalFractions. Note that we will not be using the PostProcess method in this tutorial so it will be left empty.

Setting up UserPointFunction

Next, we fill in the terminal constraints and variable bounds in the “PointObj” class which uses the UserPointFunction as its base class. Note the order of the elements in both initStateVec and finalStateVec are the same as what is defined in the Driver class. We have 8 constraints to set up for the initial and final times as well as the initial and final states. We will put these 8 constraints in the algFunctions Rvector. The order of the constraints in this Rvector do not matter. The function type for constraints is ALGEBRAIC. In the bounds’ variables for the constraints, equality constraints are set by setting the lower and upper bounds to the same value, while inequality constraints contain different values. If a lower bound is higher than an upper bound, CSALT will throw an error at runtime. Note that we must specify whether the bound we are setting is of type LOWER or UPPER. Finally, the objective function is set in this class under the function type COST. Note that the cost function must be of the Rvector data type, and not only Real.

```
// EvaluateFunctions() is where the objective, constraints,
// and bounds are set
void PointObj::EvaluateFunctions() {
    // Get initial and final states
    Rvector initStateVec = GetInitialStateVector(0);
    Rvector finalStateVec = GetFinalStateVector(0);

    // Get initial and final times
    Real initTime = GetInitialTime(0);
    Real finalTime = GetFinalTime(0);

    // Set Bounds for initial/final times or states
    Rvector algFunctions(8);
    algFunctions(0) = initTime;
    algFunctions(1) = finalTime;
    algFunctions(2) = initStateVec(0);
    algFunctions(3) = initStateVec(1);
    algFunctions(4) = initStateVec(2);
    algFunctions(5) = finalStateVec(0);
    algFunctions(6) = finalStateVec(1);
    algFunctions(7) = finalStateVec(2);

    Rvector lowerBound(8, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, -10.0, -10.0);
    Rvector upperBound(8, 0.0, 100.0, 0.0, 0.0, 0.0, 1.0, 10.0, 0.0);
    SetFunctions(ALGEBRAIC, algFunctions);
    SetFunctionBounds(ALGEBRAIC, LOWER, lowerBound);
    SetFunctionBounds(ALGEBRAIC, UPPER, upperBound);

    // Set cost function
    Rvector costFunction(1);
    costFunction(0) = finalTime;
    SetFunctions(COST, costFunction);
}

// EvaluateJacobians() is where constraints and bounds are set on
// the derivatives of the dynamics (often not used)
void PointObj::EvaluateJacobians() {
}
```

Setting up UserPathFunction

First, input the state equations into the EvaluateFunctions() method of “PathObj,”, which uses UserPathFunction as its base class. We obtain the state and control variable vectors with the GetStateVectore() and GetControlVector() methods respectively. Note the structures of these Rvectors are the same as what was defined in the Driver class. In this class we assign the first element of the state vector, X, to be the horizontal position, x, of the bead, the second element the vertical position, y, and

the third element the speed, v. The single control angle, u, is the first and only element of the control vector, U. Once the equations of motion are written we combine them in a Rvector data type with 3 elements named dynamics. The dynamics are the set with the SetFunctions() method where DYNAMICS is the function type.

```
// EvaluateFunctions() is where the dynamics and path constraints
// are set
void PathObj::EvaluateFunctions() {
    // Get states and controls for the given segment
    Rvector X = GetStateVector();
    Rvector U = GetControlVector();

    // Constants
    Real gravity = -32.174;

    // States and controls
    Real x = X(0);
    Real y = X(1);
    Real v = X(2);
    Real u = U(0);

    // State equations
    Real xdot = v * sin(u);
    Real ydot = v * cos(u);
    Real vdot = gravity * cos(u);

    // Set dynamics here
    Rvector dynamics(3, xdot, ydot, vdot);
    SetFunctions(DYNAMICS, dynamics);
}
```

```
void PathObj::EvaluateJacobians() {
    Rvector X = GetStateVector();
    Rvector U = GetControlVector();

    Real gravity = -32.174;

    Real x = X(0);
    Real y = X(1);
    Real v = X(2);
    Real u = U(0);

    Integer n = 3;          // number of states
    Integer m = 1;          // number of controls

    // Create state jacobian where n is the number of states. Automatically
    // is set to zero initially.
    Rmatrix jac(n,n);

    jac(0, 2) = sin(u);
    jac(1, 2) = cos(u);

    // Create and fill in control jacobian where m is the number of controls.
    Rmatrix ujac(n,m);

    ujac(0, 0) = v * cos(u);
    ujac(1, 0) = -v * sin(u);
    ujac(2, 0) = -gravity * sin(u);

    // Create and fill in time jacobian.
    Rmatrix tjac(n, 1);

    // Set Jacobians
    SetJacobian(DYNAMICS, STATE, jac);
    SetJacobian(DYNAMICS, CONTROL, ujac);
    SetJacobian(DYNAMICS, TIME, tjac);
}
```

Next we assign the state, control, and time Jacobians in the EvaluateJacobians() method of the PathObj class. Note that Rvector and Rmatrix class data types initially assign all elements to zero using the constructor e.g.:

Rvector vec(5);

Rmatrix mat(3, 3);

The Jacobians are all Rmatrix data types with the rows (first argument) corresponding the equation of motion, and the columns (second argument) the variable with which the derivative is taken. Note that rows and columns start at index zero.

For this first tutorial problem, comment out or delete the “OcfObj1” class as we are not using OptimalControlFunction capabilities right now.

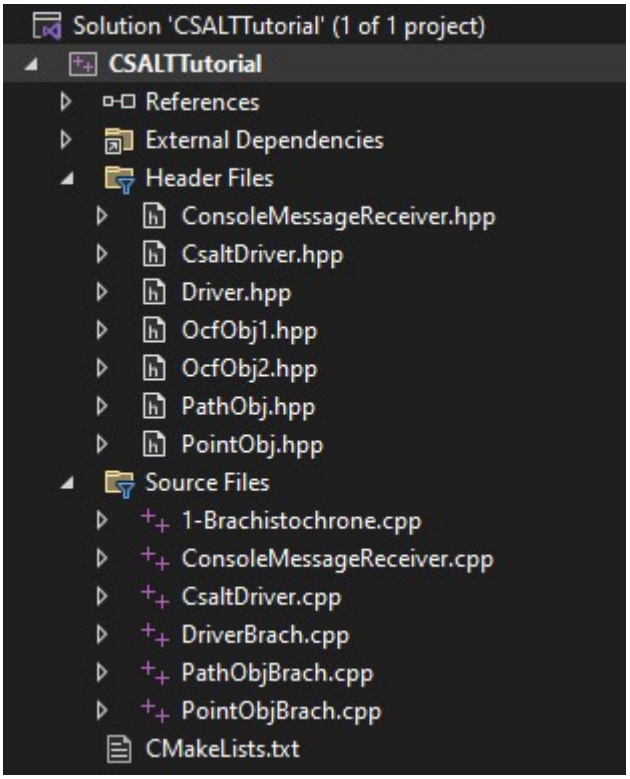
Main Function

In the main() function, located in 1-Brachistochrone.cpp, enter the name of the problem, e.g. Brachistochrone:

```
Driver* driver = new Driver("Brachistochrone");
```

Building and Running the Problem

Now that we’ve finished setting up the problem it’s time to build the solution. In Visual Studio, you should have the following files showing in the Header Files and Source Files.



Next click Build -> Build Solution (or F7). Upon successful compilation the following will be output in the console:

```
========== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ==========
========== Build started at 10:43 AM and took 04.953 seconds =========
```

If there are any unresolved or reported issues from Visual Studio, go back through the tutorial and see if you made any errors or mistakes.

To run the program, click Debug -> Start Without Debugging (or Ctrl + F5). Upon running the program, check if the SNOPT output reports the problem solved:

SNOPTA EXIT 0 -- finished successfully
SNOPTA INFO 1 -- optimality conditions satisfied

The states and controls are output to a *.och file in the BUILD\csaltTutorials folder. Likewise, the SNOPT diagnostic output is in a *Data.txt file in the same folder.

NOTE: If you get an error that states “The code execution cannot proceed because <*filename*> was not found. Reinstalling the program may fix this problem.” You are missing some application extension files or dynamic shared library file. These files can be found in \Application\bin if you have already built GMAT, or in \bin of a downloaded version of GMAT. These files will need to be copied over to the \bin folder used for CSALT.

Brachistochrone Problem with Optimal Control Function

Contents

Problem Formulation 30
Driver 30
UserPointFunction 31
UserPathFunction 31
OptimalControlFunction Class 32
Main Function 37
Building and Running the Problem 37

Problem Formulation

Same problem formulation as the first tutorial, Brachistochrone Problem, however now we will make use of the optional OptimalControlFunction class.

Driver

The Driver class setup is mostly the same as the first tutorial. The only difference is we must add the two OptimalControlFunctions we create to the CsaltDriver class. We add them by pushing them back in the ocfObjects vector data type located in the SetPointPathAndProperties() method. We must do this for all OptimalControlFunctions. Note that the order we assign them to ocfObjects does not matter. A name must also be declared for each OptimalControlFunction as the only argument to the constructor.

```
void Driver::SetPointPathAndProperties() {
    pathObject = new PathObj();
    pointObject = new PointObj();
    // Add any optional OptimalControlFunction classes to the ocfObjects vector
    // variable type. Order does not matter
    ocfObjects.push_back(new OcfObj1("BrachOCF1"));     // Time constraints
    ocfObjects.push_back(new OcfObj2("BrachOCF2"));     // State constraints

    // Set CSALT and SNOPT-related parameters in this method
    // See CsaltDriver.hpp for some of the available options
    maxMeshRefinementCount = 25;
    feasibilityTolerances[0] = 1.0e-6;
    optimizationMode[0] = "Minimize";
}
```

In this case, the OptimalControlFunction inherited classes are named OcfObj1 and OcfObj2. You can edit the names of these classes, and/or add additional OptimalControlFunction classes in the OcfObj1.hpp and OcfObj2.hpp header files. Note that we will not be using the PostProcess method in this tutorial so it will be left empty.

```
// Optional class(es) to separate constraints and let CSALT take care of
// the order and indexing. Useful for multi-phase problems with many constraints
class OcfObj1 : public OptimalControlFunction
{
public:
    // Basic constructor where we set OptimalControlFunction parameters and constraint bounds
    OcfObj1(std::string funcName) : OptimalControlFunction(funcName) {
        setupOCF();
    }

    // Basic destructor
    ~OcfObj1() {}

    // Required method to setup the OCF
    virtual void setupOCF();
    // Required method to add constraints
    virtual Rvector EvaluateFunctions();
    // Required method to scale if desired
    virtual void ScaleFunctionBounds();
};
```

UserPointFunction

The only thing to set up with the UserPointFunction is the objective, which is the same as the first tutorial.

```
// EvaluateFunctions() is where the objective, constraints,
// and bounds are set
void PointObj::EvaluateFunctions() {
    // Set cost function
    Real finalTime = GetFinalTime(0);
    Rvector costFunction(1);
    costFunction(0) = finalTime;
    SetFunctions(COST, costFunction);
}

// EvaluateJacobians() is where constraints and bounds are set on
// the derivatives of the dynamics (often not used)
void PointObj::EvaluateJacobians() {
}
```

UserPathFunction

Same UserPathFunction as the first tutorial, Brachistochrone Problem.

OptimalControlFunction Class

We can use the optional OptimalConrolFunction class to set constraints in lieu of the UserPointFunction class. We are also allowed to have as many OptimalControlFunction classes as we want. They can be useful for large multi-phase problems with many constraints, as it can be stressing on the user to bookkeep everything. The OptimalControlFunction class automatically maintains the structure of all its constraints, adding to the quality of life for the user.

The CSALTTempalte.cpp source code has an example class, OcfObj1, that inherits the OptimalControlFunction class and has the necessary methods and variables used. This tutorial will make use of two OptimalControlFunctions to split apart the time and the state constraints. Note that all constraints could be used in only one OptimalControlFunction, but we are splitting up the time and the state to show how to use more than one.

The first OptimalControlFunction will have the time constraints. First look at the setupOCF() method. The variable numPoints is the number of points where constraints are enforced on any defined phase. The numFunctions variable declares how many constraints there will be on all the points. The phaseDependencies variable is an IntegerArray data type that indicates *which phase* there are constraints for each point. Available values are up to the number of phases defined in the problem. The pointDependencies variable is an IntegerArray that indicates *where* on the phase, given by phaseDependencies, the points are located. Available values are:

0. Beginning of phase
1. End of phase
2. All points in phase (path constraint)

In this tutorial, we set numPoints to 2, since we have a time constraint at the beginning of the first (and only) phase, and a time constraint at the end of the phase. Likewise, numFunctions is 2 since we have only two constraints. We set phaseDependencies to {0, 0} since both points are in the same first phase. Finally, we set pointDependencies to {0, 1} since the initial time constraint is at the beginning of the phase, and the final time constraint is at the end of the phase.

The variables stateDepMap (for state variables), controlDepMap (for control variables), timeDepMap (for the time variable), and paramDepMap (for static parameter variables) are logical arrays that must be filled in to let the OptimalControlFunction know how many of each type of constraint there is. Since we only have two time variables, only the timeDepMap gets filled in with true values.

```
// Basic constructor where we set OptimalControlFunction parameters and constraint bounds
void OcfObj1::setupOCF() {
    // Declare the number of points where constraints are enforced on the phases
    numPoints = 2;

    // Declare the number of functions for the points in question
    numFunctions = 2;

    // The phaseDependencies variable indicates which phase the constraints apply for
    // each point. Available values are up to the number of phases in the problem
    // (index zero-based).
    IntegerArray phaseDependencies = {0, 0};

    // The pointDependencies variable indicates where on the phase given by phaseDependencies
    // the points are located. Available values are:
    // 0 - beginning of phase
    // 1 - end of phase
    // 2 - all points in phase (path constraint)
    IntegerArray pointDependencies = {0, 1};

    // Resize variables for state, control, time, and parameter dependency mapping
    stateDepMap.resize(numPoints);
    controlDepMap.resize(numPoints);
    timeDepMap.resize(numPoints);
    paramDepMap.resize(numPoints);

    // Loop through boundary points and set dependencies
    for (Integer i = 0; i < numPoints; ++i)
    {
        timeDepMap.at(i) = true;
        stateDepMap.at(i) = false;
        controlDepMap.at(i) = false;
        paramDepMap.at(i) = false;
    }
```

The rest of the setupOCF() method sets all the variables with necessary methods to employ the OptimalControlFunction, as well as sets the bounds on the constraints.

```
    // Initialize the OptimalControlFunction and set desired parameters
    Initialize();
    SetNumPhases(numPoints);
    SetNumFunctions(numFunctions);
    SetPhaseDependencies(phaseDependencies);
    SetPointDependencies(pointDependencies);
    SetStateDepMap(stateDepMap);
    SetControlDepMap(controlDepMap);
    SetTimeDepMap(timeDepMap);
    SetParamDepMap(paramDepMap);

    // Set lower and upper bounds
    Rvector lowerBound(numFunctions, 0.0, 0.0);
    Rvector upperBound(numFunctions, 0.0, 100.0);
    lowerBounds = lowerBound;
    upperBounds = upperBound;
}
```

The next required method to fill in is EvaluateFunctions(). This is where the constraints are actually set. We set the first constraint to timeData[0][0], where the first index is the point the constraint is being applied, and the second is the location in the array. Since everything has zero base indices, 0 in the first index is the first point, and this point only has one element which is the initial time at the beginning of the phase. The second constraint timeData[1][0], indicates we are using the second point with the first index set to 1, and using the only element which is the final time at the end of the phase.

The last required method is the ScaleFunctionBounds() if you desire to do so. The optional method ValidateFunctionBounds() is put in this method to check that the bounds are the same size and that they have valid Real values. It is often better to scale or nondimensionalize the entire problem instead of only scaling the variables and constraints of a problem with proper units.

```
Rvector OcfObj1::EvaluateFunctions() {
    // Set the constraints. First index indicates the desired point
    // Second index indicates location in the array
    Rvector algF(numFunctions);
    algF(0) = timeData[0][0];
    algF(1) = timeData[1][0];

    return algF;
}

// Required function to scale the constraint function bounds if desired
void OcfObj1::ScaleFunctionBounds() {
    // Check that bounds are same size and have valid values
    ValidateFunctionBounds();
}
```


```
Rvector OcfObj1::EvaluateFunctions() {
    // Set the constraints. First index indicates the desired point
    // Second index indicates location in the array
    Rvector algF(numFunctions);
    algF(0) = timeData[0][0];
    algF(1) = timeData[1][0];

    return algF;
}

// Required function to scale the constraint function bounds if desired
void OcfObj1::ScaleFunctionBounds() {
    // Check that bounds are same size and have valid values
    ValidateFunctionBounds();
}
```


Now we fill in the second OptimalControlFunction for the state constraints which we call OcfObj2. The variable numPoints is still 2 since we only have state constraints at the beginning and the end of the first phase. The number of functions is 6 as there are six boundary conditions. The phaseDependencies and pointDependencies variables are the same with {0, 0} and {0, 1} respectively. Only the stateDepMap will have 2 values of true.

```
// Basic constructor where we set OptimalControlFunction parameters and constraint bounds
void OcfObj2::setupOCF() {
    // Declare the number of points where constraints are enforced on the phases
    numPoints = 2;

    // Declare the number of functions for the points in question
    numFunctions = 6;

    // The phaseDependencies variable indicates which phase the constraints apply for
    // each point. Available values are up to the number of phases in the problem
    // (index zero-based).
    IntegerArray phaseDependencies = { 0, 0 };

    // The pointDependencies variable indicates where on the phase given by phaseDependencies
    // the points are located. Available values are:
    // 0 - beginning of phase
    // 1 - end of phase
    // 2 - all points in phase (path constraint)
    IntegerArray pointDependencies = { 0, 1 };

    // Resize variables for state, control, time, and parameter dependency mapping
    stateDepMap.resize(numPoints);
    controlDepMap.resize(numPoints);
    timeDepMap.resize(numPoints);
    paramDepMap.resize(numPoints);

    // Loop through boundary points and set dependencies
    for (Integer i = 0; i < numPoints; ++i)
    {
        timeDepMap.at(i) = false;
        stateDepMap.at(i) = true;
        controlDepMap.at(i) = false;
        paramDepMap.at(i) = false;
    }
```

Next set the bounds on the state constraints.

```
// Initialize the OptimalControlFunction and set desired parameters
Initialize();
SetNumPhases(numPoints);
SetNumFunctions(numFunctions);
SetPhaseDependencies(phaseDependencies);
SetPointDependencies(pointDependencies);
SetStateDepMap(stateDepMap);
SetControlDepMap(controlDepMap);
SetTimeDepMap(timeDepMap);
SetParamDepMap(paramDepMap);

// Set lower and upper bounds
Rvector lowerBound(numFunctions, 0.0, 0.0, 0.0, 1.0, -10.0, -10.0);
Rvector upperBound(numFunctions, 0.0, 0.0, 0.0, 1.0, 10.0, 0.0);
lowerBounds = lowerBound;
upperBounds = upperBound;
```

Finally, we set the constraints themselves in the EvaluateFunctions() method. The first index of stateData refers to which point you are using, and the second index is the state at that point. Note that the second index of stateData refers to the state, and in this case 0 = x, 1 = y, and 2 = v.

```
// Required function to evaluate constraints at the desired points in the phases
Rvector OcfObj2::EvaluateFunctions() {

    // Set the constraints. First index indicates the desired point
    // Second index indicates which state in the array
    Rvector algF(numFunctions);
    algF(0) = stateData[0][0];
    algF(1) = stateData[0][1];
    algF(2) = stateData[0][2];
    algF(3) = stateData[1][0];
    algF(4) = stateData[1][1];
    algF(5) = stateData[1][2];

    return algF;
}

// Required function to scale the constraint function bounds if desired
void OcfObj2::ScaleFunctionBounds() {
    // Check that bounds are same size and have valid values
    ValidateFunctionBounds();
}
```

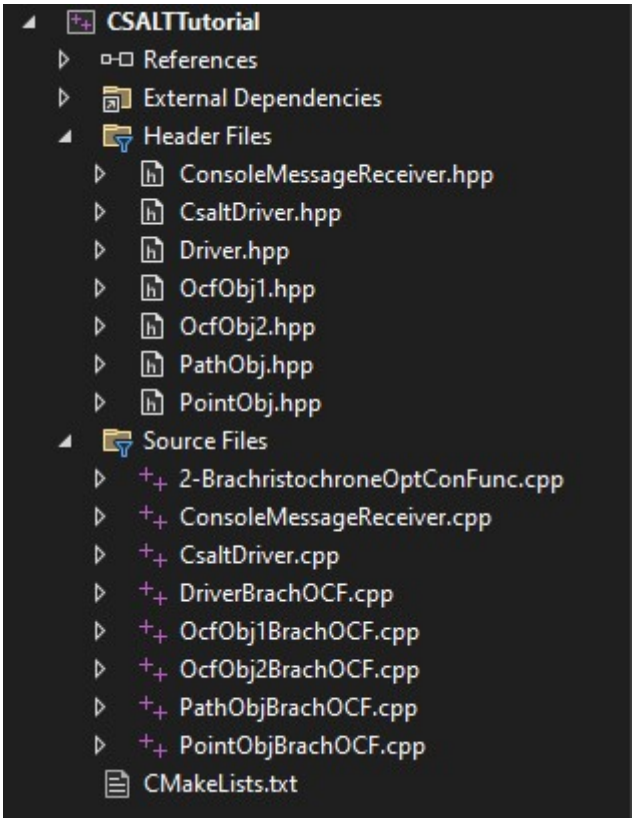
Main Function

In the main() function, located in 2-BrachristochroneOptConFunc.cpp, enter the name of the problem, e.g. BrachistochroneOptConFunc:

```
Driver* driver = new Driver("BrachistochroneOptConFunc");
```

Building and Running the Problem

Now that we've finished setting up the problem it's time to build the solution. In Visual Studio, you should have the following files showing in the Header Files and Source Files.



Compile and run the second tutorial the same way you did for the first tutorial. Upon completion check to verify if the results are the same (or very close to) the first tutorial, as all we did was change the location of the constraints in the problem.

Conway Low Thrust Spiral

Contents

Problem Formulation 39
Driver 40
UserPointFunction 41
Setting up UserPathFunction 41
Main Function 42
Building and Running the Problem 42

Problem Formulation

Maximize the final orbital energy of a planar low thrust spiral starting at a circular orbit. All quantities are nondimensional.

$$Max\ J = \frac{1}{2}\left(u_f^2 + v_f^2\right) - \frac{1}{r_f}$$

Subject to:

$$\dot{r} = u$$

$$\dot{\theta} = \frac{v}{r}$$

$$\dot{u} = \frac{v^2}{r} - \frac{1}{r^2} + A\sin\alpha$$

$$\dot{v} = -\frac{uv}{r} + A\cos\alpha$$

Initial conditions:

$$t_0 = 0$$

$$r(t_0) = 1.1$$

$$\theta(t_0) = 0$$

$$u(t_0) = 0$$

$$v(t_0) = \frac{1}{\sqrt{1.1}}$$

Variable bounds:

$$-10 \le r(t) \le 10$$

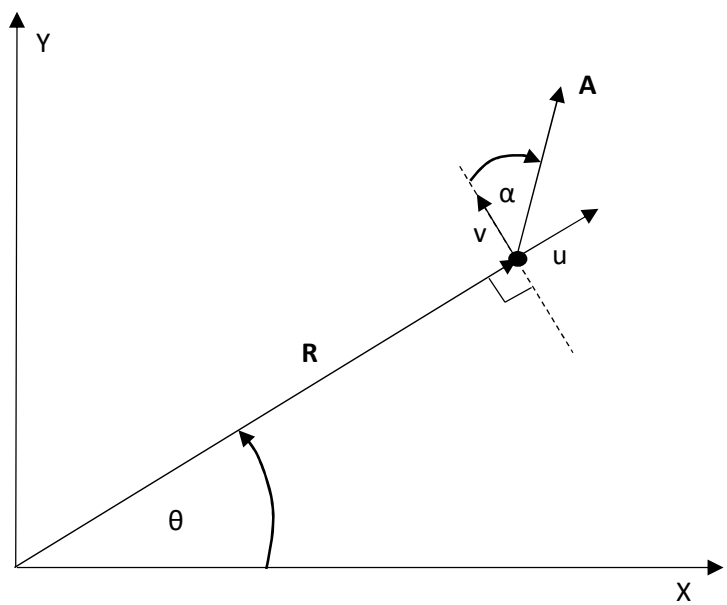
$$-12\pi \leq \theta(t) \leq 12\pi$$

$$-10 \leq u(t) \leq 10$$

$$-10 \leq v(t) \leq 10$$

$$-10 \leq \alpha(t) \leq 10$$

With constant A = 0.01. See picture below for visualization. Note that the orthogonal components [u, v] make up the velocity vector, |**R**| = r, and **A** is the constant acceleration (thrust) oriented by the control, α.



Driver

We will setup the mesh variables a bit differently in the Driver class of this tutorial. This time we will use a mesh interval spanning 5 equidistant points between [-1, 1], with 12 points between each interval. We will also make use of the constant π which is included in the GmatMathConstants namespace with GmatMathConstants::PI. Additionally, our goal is to maximize the cost function which can be accomplished one of two ways. The first way is to set the first element of the string array optimizationMode to "Maximize" in the SetPointPathAndProperties() method, or we can minimize the negative of the original cost function. The below example code sets opimizationMode to "Maximize". For the initialGuessMode we will use "LinearUnityControl". There is no inherent reason to use this initial guess mode, but it has shown to work well with trial-and-error. Note that we will not be using the PostProcess method in this tutorial so it will be left empty.

```
// SetPointPathAndProperties() method is where the Path, Point, and
// OptimalControlFunction classes will be instantiated.
void Driver::SetPointPathAndProperties() {
    pathObject = new PathObj();
    pointObject = new PointObj();

    // Set CSALT and SNOPT-related parameters in this method
    // See CsaltDriver.hpp for some of the available options
    maxMeshRefinementCount = 25;
    feasibilityTolerances(0) = 1.0e-6;
    optimizationMode[0] = "Maximize";
}

void Driver::SetupPhases() {
    //Set mesh properties
    RadauPhase* phase1 = new RadauPhase();
    // Initial guess modes: "LinearNoControl", "LinearUnityControl",
    // "LinearCoast", "OCHFile", "GuessArrays"
    std::string initialGuessMode = "LinearUnityControl";
    Rvector meshIntervalFractions(5, -1.0, -0.5, 0.0, 0.5, 1.0);

    IntegerArray meshIntervalNumPoints = { 12,12,12,12 };

    //Set time properties
    Real timeLowerBound = 0.0;
    Real timeUpperBound = 100.0;
    Real initialGuessTime = 0.0;
    Real finalGuessTime = 50.0;

    //Set state properties
    Integer numStateVars = 4;
    Rvector stateLowerBound(4, 0.0, -6.0 * 2.0 * GmatMathConstants::PI, -10.0, -10.0);
    Rvector initialGuessState(4, 1.1, 0.0, 0.0, 1.0 / sqrt(1.1));
    Rvector finalGuessState(4, 5.0, 3.0 * 2.0 * GmatMathConstants::PI, 1.0, 1.0);
    Rvector stateUpperBound(4, 10.0, 6.0 * 2.0 * GmatMathConstants::PI, 10.0, 10.0);

    //Set control properties
    Integer numControlVars = 1;
    Rvector controlUpperBound(1, 10.0);
    Rvector controlLowerBound(1, -10.0);
```

UserPointFunction

The setup of UserPointFunction is very similar to the first tutorial. See if you can fill this class in on your own. The things to fill in are the initial time constraint, final time constraint, initial state constraints, and the objective function.

Setting up UserPathFunction

The setup of the UserPathFunction is very similar to the first tutorial. See if you can fill this class in on your own. Frequent sources of error can occur in the derivation and filling in of the Jacobians, so be sure to go over them more than once.

Main Function

In the main() function, included in 3-ConwayLowThrust.cpp, enter the name of the problem, e.g. ConwayLowThrust:

```
Driver* driver = new Driver("ConwayLowThrust");
```

Building and Running the Problem

Compile and run the third tutorial the same way you did for the first and second tutorials.

Orbit Raising Using Multiple Phases and an OptimalControlFunction

Contents

Problem Formulation 43
Setting up UserPathFunction 48
UserPointFunction 45
Linking Phases with a OptimalControlFunction 50
Driver 45
Building and Running the Problem 51

Problem Formulation

Maximize the final radius of a planar low thrust spiral from one circular orbit to another in a fixed amount of time. All quantities are nondimensional.

$$Max\ J = r(t_f)$$

Subject to:

$$\dot{r} = u$$

$$\dot{\theta} = \frac{v}{r}$$

$$\dot{u} = \frac{v^2}{r} - \frac{1}{r^2} + \frac{T}{m}u_r$$

$$\dot{v} = -\frac{uv}{r} + \frac{T}{m}u_\theta$$

$$\dot{m} = m_{dot}$$

Path constraint:

$$u_r^2 + u_\theta^2 = 1$$

Initial conditions:

$$t_0 = 0$$

$$r(t_0) = 1$$

$$\theta(t_0) = 0$$

$$u(t_0) = 0$$

$$v(t_0) = 1$$

$$m(t_0) = 1$$

Final conditions:

$$t_f = 3.32$$

$$u(t_f) = 0$$

$$v(t_f) = \sqrt{\frac{1}{r(t_f)}}$$

Variable bounds on both phases:

$$0 \leq t \leq 3.32$$

$$0.5 \leq r(t) \leq 5$$

$$-\pi \leq \theta(t) \leq \pi$$

$$-10 \leq u(t) \leq 10$$

$$-10 \leq v(t) \leq 10$$

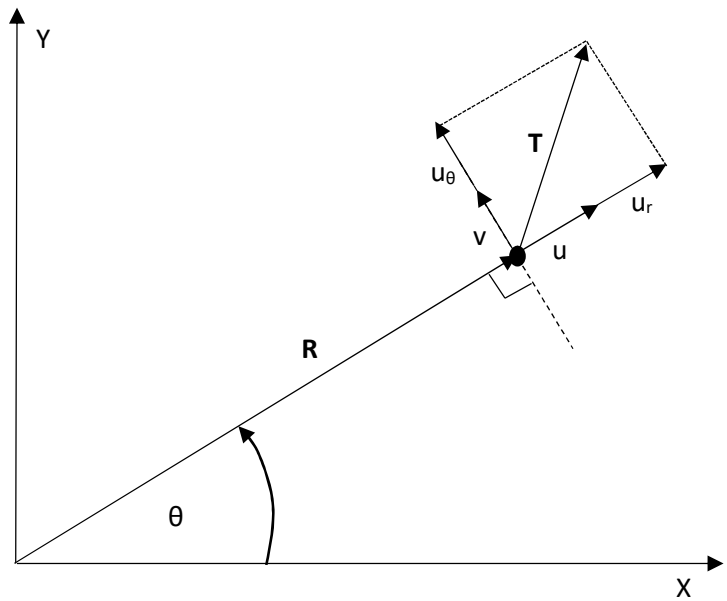
$$0.1 \leq m(t) \leq 1$$

$$0.1 \leq m(t) \leq 1$$

$$-10 \leq u_r(t) \leq 10$$

$$-10 \leq u_\theta(t) \leq 10$$

With constants T = 0.1405 and m_{dot} = -0.0749. See picture below for visualization. Note that the orthogonal components [u, v] make up the velocity vector, |**R**| = r, and the thrust vector **T** has components **T** = T*[u_r, u_θ]



Driver

The Driver class for the fourth tutorial will increase in size due to there now being more than one phase. Remember to push back the OptimalControlFunction class to the ocfObjects vector, and that we are maximizing a quantity in our optimization. The mesh for the two phases will just be the endpoints on [-1, 1], with 7 points between them.

```
// SetPointPathAndProperties() method is where the Path, Point, and
// OptimalControlFunction classes will be instantiated.
void Driver::SetPointPathAndProperties() {
    pathObject = new PathObj();
    pointObject = new PointObj();
    // Add any optional OptimalControlFunction classes to the ocfObjects vector
    // variable type. Order does not matter
    ocfObjects.push_back(new OcfObj1("OcfObj1"));

    // Set CSALT and SNOPT-related parameters in this method
    // See CsaltDriver.hpp for options
    maxMeshRefinementCount = 25;
    feasibilityTolerances(0) = 1.0e-6;
    optimizationMode[0] = "Maximize";
}

void Driver::SetupPhases() {
    //Set mesh properties
    RadauPhase* phase1 = new RadauPhase();
    // Initial guess modes: "LinearNoControl", "LinearUnityControl",
    // "LinearCoast", "OCHFile", "GuessArrays"
    std::string initialGuessMode = "LinearUnityControl";
    Rvector meshIntervalFractions(2, -1.0, 1.0);
    IntegerArray meshIntervalNumPoints = { 7 };
```

After we setup the bounds, we push back the first phase into the phaseList vector and begin the setting up the second phase

```
// Set phase options
phase1->SetInitialGuessMode(initialGuessMode);
// Needed if initialGuessMode is "GuessArrays"
// phase1->SetInitialGuessArrays(
phase1->SetNumStateVars(numStateVars);
phase1->SetNumControlVars(numControlVars);
phase1->SetMeshIntervalFractions(meshIntervalFractions);
phase1->SetMeshIntervalNumPoints(meshIntervalNumPoints);
phase1->SetStateLowerBound(stateLowerBound);
phase1->SetStateUpperBound(stateUpperBound);
phase1->SetStateInitialGuess(initialGuessState);
phase1->SetStateFinalGuess(finalGuessState);
phase1->SetTimeLowerBound(timeLowerBound);
phase1->SetTimeUpperBound(timeUpperBound);
phase1->SetTimeInitialGuess(initialGuessTime);
phase1->SetTimeFinalGuess(finalGuessTime);
phase1->SetControlLowerBound(controlLowerBound);
phase1->SetControlUpperBound(controlUpperBound);

// Set phase to phaseList vector data type
phaseList.push_back(phase1);

RadauPhase* phase2 = new RadauPhase();
// Initial guess modes: "LinearNoControl", "LinearUnityControl",
// "LinearCoast", "OCHFile", "GuessArrays"
std::string initialGuessMode2 = "LinearUnityControl";
Rvector meshIntervalFractions2(2, -1.0, 1.0);
IntegerArray meshIntervalNumPoints2 = { 7 };
```

All variable and control bounds are the same for both phases as outlined in the Problem Formulation section. Once the bounds for the second phase are set up, we push back the second phase to the phaseList vector. Note that we will not be using the PostProcess method in this tutorial so it will be left empty.

```
// Set phase options
phase2->SetInitialGuessMode(initialGuessMode2);
// Needed if initialGuessMode is "GuessArrays"
// phase2->SetInitialGuessArrays(
phase2->SetNumStateVars(numStateVars2);
phase2->SetNumControlVars(numControlVars2);
phase2->SetMeshIntervalFractions(meshIntervalFractions2);
phase2->SetMeshIntervalNumPoints(meshIntervalNumPoints2);
phase2->SetStateLowerBound(stateLowerBound2);
phase2->SetStateUpperBound(stateUpperBound2);
phase2->SetStateInitialGuess(initialGuessState2);
phase2->SetStateFinalGuess(finalGuessState2);
phase2->SetTimeLowerBound(timeLowerBound2);
phase2->SetTimeUpperBound(timeUpperBound2);
phase2->SetTimeInitialGuess(initialGuessTime2);
phase2->SetTimeFinalGuess(finalGuessTime2);
phase2->SetControlLowerBound(controlLowerBound2);
phase2->SetControlUpperBound(controlUpperBound2);

// Set phase to phaseList vector data type
phaseList.push_back(phase2);
```

UserPointFunction

The UserPointFunction will contain the constraints at the endpoints of the problem, that is, the beginning of the first phase and the end of the second phase. The constraints linking the time and states together will be handled in an OptimalControlFunction class. In the example below, we have obtained the initial time and state of the first phase with GetInitialTime(0) and GetInitialStateVector(0), where the 0 index refers to the first phase. The final time and state of the second phase is obtained with GetFinalTime(1) and GetFinalStateVector(1), where the 1 index refers to the second phase. We then set the appropriate constraints. In this example the state vector is structured with the following elements:

0. r
1. θ
2. u
3. v
4. m

and the control vector:

0. u_r
1. u_θ

Note that the constraint

$$v(t_f) = \sqrt{\frac{1}{r(t_f)}}$$

must be reformulated as

$$\sqrt{\frac{1}{r(t_f)}} - v(t_f) = 0 \qquad \text{or} \qquad v(t_f) - \sqrt{\frac{1}{r(t_f)}} = 0$$

so that the lower and upper bounds contain a constant 0 and not a time-varying state.

```
// EvaluateFunctions() is where the objective, constraints,
// and bounds are set
void PointObj::EvaluateFunctions() {
    // Get initial and final states
    Rvector X10 = GetInitialStateVector(0);
    Rvector X2f = GetFinalStateVector(1);

    // Get initial and final times
    Real T10 = GetInitialTime(0);
    Real T2f = GetFinalTime(1);

    // Set Bounds for initial/final times or states
    Rvector algFunctions(9);
    algFunctions(0) = T10;
    for (Integer i = 0; i < 5; ++i)
        algFunctions(i + 1) = X10(i);
    algFunctions(6) = T2f;
    algFunctions(7) = X2f(2);
    algFunctions(8) = sqrt(1.0 / X2f(0)) - X2f(3);

    Rvector lowerBound(9, 0.0, 1.0, 0.0, 0.0, 1.0, 1.0, 3.32, 0.0, 0.0);
    Rvector upperBound(9, 0.0, 1.0, 0.0, 0.0, 1.0, 1.0, 3.32, 0.0, 0.0);
    SetFunctions(ALGEBRAIC, algFunctions);
    SetFunctionBounds(ALGEBRAIC, LOWER, lowerBound);
    SetFunctionBounds(ALGEBRAIC, UPPER, upperBound);

    // Set cost function
    Rvector costFunction(1);
    costFunction(0) = X2f(0);           // maximize final radius
    SetFunctions(COST, costFunction);
}

// EvaluateJacobians() is where constraints and bounds are set on
// the derivatives of the dynamics (often not used)
void PointObj::EvaluateJacobians() {
}
```

UserPathFunction

This tutorial problem is like the third tutorial in terms of dynamics, but different in the following ways:

- Spacecraft mass is a state variable
- A path constraint is used to keep the steering unit vector at unity
- The problem is being split up into two phases
- Using an OptimalControlFunction to link the time and state of the two phases together.

The dynamics in the UserPathFunction for this tutorial is very similar to the third tutorial except for the additional mass state variable and splitting up the control into two components of a unit vector. See if you can add these new state and controls to the dynamics. If you need additional help, consult the OrbitRaisingMultiPhaseOptConFunc.cpp source solution.

New to this tutorial, we will now add a path constraint to ensure the magnitude of the control unit vector is always unity. We can create path constraints with the SetFunctions() method with the ALGEBRAIC function type.

```
// Add path constraints
Rvector path(1), pathlb(1), pathub(1);
path(0) = ur * ur + uth * uth;
pathlb(0) = 1.0;
pathub(0) = 1.0;
SetFunctions(ALGEBRAIC, path);
SetFunctionBounds(ALGEBRAIC, LOWER, pathlb);
SetFunctionBounds(ALGEBRAIC, UPPER, pathub);
```

Note that ‘path’ is the path constraint, ‘pathlb’ is the lower bound, ‘pathub’ is the upper bound, and they must all be of the Rvector data type.

Since we have added a state variable and a control variable, the Jacobians related to the state and control variables will expand in the EvaluateJacobians() method compared to the third tutorial. We also add the Jacobian of the path constraint to this method. The composition of the path constraint Jacobian is as follows: rows correspond to the constraint number, columns correspond to the number of states, controls, and time variables. All Jacobians must be of the Rmatrix data type, so even if the Jacobian has size 1 x 1 (which is technically a scalar), it still must be declared as a Rmatrix.

```
// Add derivatives of path constraint
Rmatrix dpathx(1, n), dpathu(1, nu), dpatht(1, 1);
dpathu(0, 0) = 2.0 * ur;
dpathu(0, 1) = 2.0 * uth;
SetJacobian(ALGEBRAIC, STATE, dpathx);
SetJacobian(ALGEBRAIC, CONTROL, dpathu);
SetJacobian(ALGEBRAIC, TIME, dpatht);
```

Note that ‘n’ = 5 is the number of states, ‘nu’ = 2 is the number of controls, ‘dpathx’ is the Jacobian of the path constraint with respect to the states, ‘dpathu’ with respect to the controls, and ‘dpatht’ with respect to time. In the above example control Jacobian, ‘ur’ is the 0th index of the control vector, and ‘uth’ is the 1st index.

Linking Phases with a OptimalControlFunction

Now we link the final time and states of the first phase to the initial time and states of the second phase in an OptimalControlFunction class. This can be done in the UserPointFunction class, but it will add complexity to the size and structure of the constraint arrays. Using the OptimalControlFunction containerizes the constraints so that they are more user-friendly. You can see when you have many phases interacting with each other with multiple constraints, how the OptimalControlFunction becomes a useful utility for your problem.

```
// Basic constructor where we set OptimalControlFunction parameters and constraint bounds
void OcfObj1::setupOCF() {
    // Declare the number of points where constraints are enforced on the phases
    numPoints = 2;

    // Declare the number of functions for the points in question
    numFunctions = 6;

    // The phaseDependencies variable indicates which phase the constraints apply for
    // each point. Available values are up to the number of phases in the problem
    // (index zero-based).
    IntegerArray phaseDependencies = { 0, 1 };

    // The pointDependencies variable indicates where on the phase given by phaseDependencies
    // the points are located. Available values are:
    // 0 - beginning of phase
    // 1 - end of phase
    // 2 - all points in phase (path constraint)
    IntegerArray pointDependencies = { 1, 0 };

    // Resize variables for state, control, time, and parameter dependency mapping
    stateDepMap.resize(numPoints);
    controlDepMap.resize(numPoints);
    timeDepMap.resize(numPoints);
    paramDepMap.resize(numPoints);

    // Loop through boundary points and set dependencies
    for (Integer i = 0; i < numPoints; ++i) {
        timeDepMap.at(i) = true;
        stateDepMap.at(i) = true;
        controlDepMap.at(i) = false;
        paramDepMap.at(i) = false;
    }
```

In the above example OptimalControlFunction constructor, numPoints is set to 2. The first point is at the end of the first phase, and the second point is at the beginning of the second phase. The numFunctions variable is 6 as there is one time linkage constraint and five state linkage constraints. The phaseDependencies array is {0, 1} meaning the first point (0) is in the first phase, and the second point (1) is in the second phase (remember 0 based indexing in C++). The pointDependencies array is {1, 0} meaning the first point is at the end of the first phase, and the second point is at the beginning of the second phase. Set both timeDepMap and stateDepMap to have two values of 'true' each.

```
    // Set lower and upper bounds
    Rvector lBounds(numFunctions);
    Rvector uBounds(numFunctions);
    lowerBounds = lBounds;
    upperBounds = uBounds;
}

// Required function to evaluate constraints at the desired points in the phases
Rvector OcfObj1::EvaluateFunctions() {
    // Evaluate linkage constraints between phase1 and phase2
    // First index indicates the desired point
    // Second index indicates location in the array (timeData or stateData)
    Rvector algF(numFunctions);
    algF(0) = timeData[0][0] - timeData[1][0];              // Link time between phase1 and phase2
    algF(1) = stateData[0][0] - stateData[1][0];            // Link r between phase1 and phase2
    algF(2) = stateData[0][1] - stateData[1][1];            // Link th between phase1 and phase2
    algF(3) = stateData[0][2] - stateData[1][2];            // Link u between phase1 and phase2
    algF(4) = stateData[0][3] - stateData[1][3];            // Link v between phase1 and phase2
    algF(5) = stateData[0][4] - stateData[1][4];            // Link m between phase1 and phase2

    return algF;
}
```

We are going to have our time and state linkage constraints to have the structure phase1 – phase2 = 0. Doing this has the lower and upper bounds all be zeros, and the constraints are applied via the above example.

Main Function

In the main() function, included in 4-OrbitRaisingMultiPhaseOptConFunc.cpp, enter the name of the problem, e.g. OrbitRaisingMultiPhase:

```
Driver* driver = new Driver("OrbitRaisingMultiPhase");
```

Building and Running the Problem

Compile and run the fourth tutorial the same way you did for the previous tutorials.

DRO to L2 in the CR3BP

Contents

Problem Formulation 52
Driver 54
UserPointFunction 56
UserPathFunction 56
Main Function 56
Building and Running the Problem 57

Problem Formulation

Maximize the final mass of a low-thrust trajectory from a lunar distant retrograde orbit (DRO) to a periodic orbit around the L2 Lagrange point in the Circular-Restricted Three Body Problem (CR3BP) in a fixed time. All quantities are nondimensional.

$$Max\ J = m_f$$

Subject to:

$$\dot{x} = v_x$$

$$\dot{y} = v_y$$

$$\dot{z} = v_z$$

$$\dot{v}_x = x + 2v_y - \frac{(1-\mu)(x+\mu)}{d_1^3} - \frac{\mu(x-1+\mu)}{d_2^3} + \frac{T_U^2}{1000 * D_U}\frac{u_x}{\overline{m}}$$

$$\dot{v}_y = y - 2v_x - \frac{(1-\mu)y}{d_1^3} - \frac{\mu y}{d_2^3} + \frac{T_U^2}{1000 * D_U}\frac{u_y}{\overline{m}}$$

$$\dot{v}_z = -\frac{(1-\mu)z}{d_1^3} - \frac{\mu z}{d_2^3} + \frac{T_U^2}{1000 * D_U}\frac{u_z}{\overline{m}}$$

$$\dot{m} = -\frac{T_U u_{mag}}{I_{SP} g_0 m_{scale}}$$

where:

$$\overline{m} = m * m_{scale}$$

$$d_1 = \sqrt{(x+\mu)^2 + y^2 + z^2}$$

$$d_2 = \sqrt{(x-1+\mu)^2 + y^2 + z^2}$$

$$u_{mag} = \sqrt{u_x^2 + u_y^2 + u_z^2 + \sqrt{\epsilon}}$$

$$\mu = 0.012150585609624037$$

$$T_U = 375699.81732246041$$

$$D_U = 384747.96285603708$$

$$I_{SP} = 3000$$

$$g_0 = 9.80665$$

$$m_{scale} = 1000$$

$$\sqrt{\epsilon} = \sqrt{2.22044604925031\mathrm{e} - 16}$$

Initial conditions:

$$t_0 = 0$$

$$x(t_0) = 1.07730946$$

$$y(t_0) = 0$$

$$z(t_0) = 0$$

$$v_x(t_0) = 0$$

$$v_y(t_0) = -0.46973763$$

$$v_z(t_0) = 0$$

$$m(t_0) = 1.5$$

Final conditions:

$$t_f = 4.50492518$$

$$x(t_f) = 1.11989122$$

$$y(t_f) = -0.11468544$$

$$z(t_f) = -0.08789384$$

$$v_x(t_f) = -0.09155242$$

$$v_y(t_f) = -0.10503982$$

$$v_z(t_f) = 0.1516538$$

Variable bounds:

$$-2 < x(t) \leq 2$$

$$-2 \leq y(t) \leq 2$$

$$-2 \leq z(t) \leq 2$$

$$-2 \leq v_x(t) \leq 2$$

$$-2 \leq v_y(t) \leq 2$$

$$-2 \leq v_z(t) \leq 2$$

$$1 \leq m(t) \leq 1.5$$

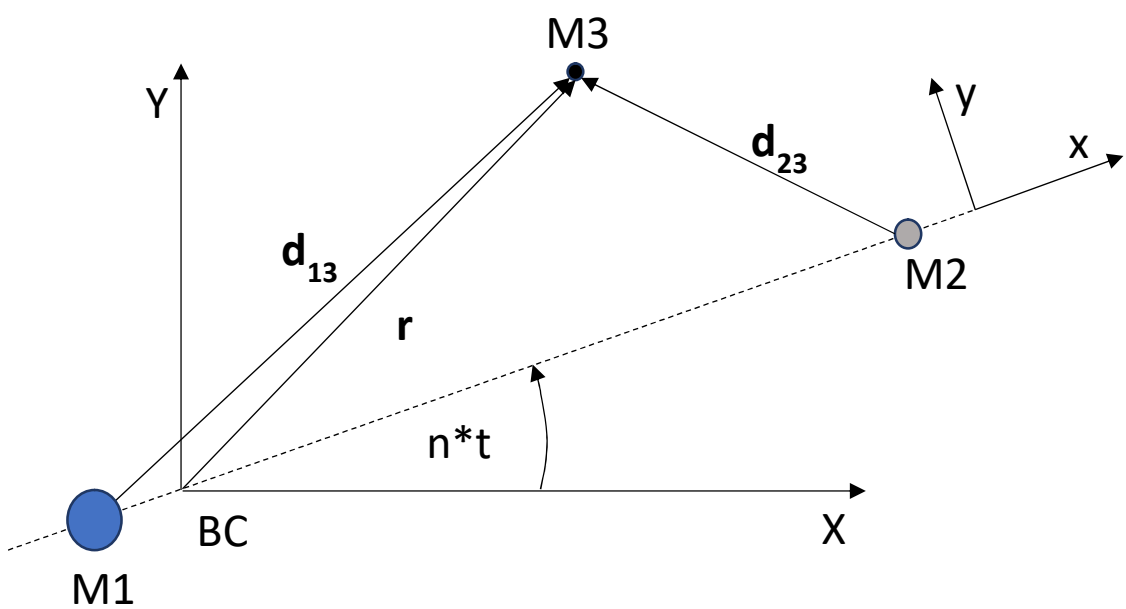
States: x, y, z, v_x, v_y, v_z, m

Controls: u_x, u_y, u_z

The $\sqrt{\epsilon} = \sqrt{2.22044604925031\mathrm{e} - 16}$ term is used for numerical stability purposes.

We divide the thrust terms in the equations of motion by 1000 to account for nondimensional units.

See picture below for visualization. The origin is the barycenter (BC) of the two primary masses M1 and M2. M3 represents the spacecraft and is assumed to be small enough to not influence the motion of the two primaries. The working frame [x, y, z] is rotating around the barycenter with x pointing from M1 to M2, and z along the direction of the angular velocity of the rotating frame (coming out of the page). The rotating frame has a constant angular rate n, corresponding to the mean motion of the two primaries. M1 and M2 stay a constant distance away from each other so that they each maintain a circular orbit around the barycenter.



Driver

This is the fifth and final capstone problem for the CSALT tutorials. The CR3BP is a higher-fidelity problem with analysis having been applied to real-world applications such as the James Webb Space

Telescope in the Sun-Earth system. The problem has only one path constraint to maintain unity in the control unit vector, but the complexity lies within the dynamics and their Jacobians.

The only new content in this tutorial is using outside data files to seed the initial guess for the optimizer. To do this we set the initialGuessMode variable to "GuessArrays", and include the SetInitialGuessArrays() method to the phase's class. Set the mesh to be 10 equidistant points spanning [-1, 1] with 5 points between each interval:

```
Rvector meshIntervalFractions(10);
// linspace(a, b, n) equivalent a + (b - a)/(n - 1) * i
for (int i = 0; i < 10; ++i)
      meshIntervalFractions(i) = -1.0 + 2.0 / (10-1) * i;

IntegerArray meshIntervalNumPoints{5, 5, 5, 5, 5, 5, 5, 5, 5};
```

```
// Set phase options
phase1->SetInitialGuessMode(initialGuessMode);
phase1->SetInitialGuessArrays(timeArray, stateArray, controlArray);
phase1->SetNumStateVars(numStateVars);
phase1->SetNumControlVars(numControlVars);
phase1->SetMeshIntervalFractions(meshIntervalFractions);
phase1->SetMeshIntervalNumPoints(meshIntervalNumPoints);
phase1->SetStateLowerBound(stateLowerBound);
phase1->SetStateUpperBound(stateUpperBound);
phase1->SetStateInitialGuess(initialGuessState);
phase1->SetStateFinalGuess(finalGuessState);
phase1->SetTimeLowerBound(timeLowerBound);
phase1->SetTimeUpperBound(timeUpperBound);
phase1->SetTimeInitialGuess(initialGuessTime);
phase1->SetTimeFinalGuess(finalGuessTime);
phase1->SetControlLowerBound(controlLowerBound);
phase1->SetControlUpperBound(controlUpperBound);

// Set phase to phaseList vector data type
phaseList.push_back(phase1);
```

In the above example, 'timeArray', 'stateArray', and 'controlArray' are the variables containing the initial guess. The timeArray variable is a Rvector data type while stateArray and controlArray are Rmatrix data types. The files t_guess.dat, x_guess.dat, and u_guess.dat contain the initial guess for the time, state, and controls respectively. The location of these files is in BUILD\csaltTutorials. The below example uses C-style syntax to read in the data, but you may use your preferred method. Note that there are 150 points of data in the initial guess data files. Depending on whether you run the executable from Visual Studio, or from Windows itself, the t_guess.dat, x_guess.dat, and u_guess.dat will need to be in either the BUILD\CsaltTutorials directory (Visual Studio) or the BUILD\CsaltTutorials\bin directory (Windows).

```
// Obtain initial guess for time, state, and controls from data files
Integer n = 150;
Rvector timeArray(n);
Rmatrix stateArray(n, 7);
Rmatrix controlArray(n, 3);
FILE *fidt, *fidx, *fidu;
fidt = fopen("t_guess.dat", "r");
fidx = fopen("x_guess.dat", "r");
fidu = fopen("u_guess.dat", "r");
for (int i = 0; i < n; ++i) {
    (void)fscanf(fidt, "%lf", &timeArray(i));
    (void)fscanf(fidx, "%lf %lf %lf %lf %lf %lf %lf", &stateArray(i, 0),
        &stateArray(i, 1), &stateArray(i, 2), &stateArray(i, 3),
        &stateArray(i, 4), &stateArray(i, 5), &stateArray(i, 6));
    (void)fscanf(fidu, "%lf %lf %lf", &controlArray(i, 0),
        &controlArray(i, 1), &controlArray(i, 2));
}
fclose(fidt);
fclose(fidx);
fclose(fidu);
```

In the SetPointPathAndProperties() method, add the following settings:

```
maxMeshRefinementCount = 6;
majorOptimalityTolerances = Rvector(1, 1.0e-2);
feasibilityTolerances = Rvector(1, 1.0e-5);
```

Note that we will not be using the PostProcess method in this tutorial so it will be left empty.

UserPointFunction

All initial and final times and states are specified except the final mass which we are maximizing. We are not using an OptimalControlFunction, so all constraints will be included in the UserPointFunction.

UserPathFunction

Using the knowledge you've gained from the previous four tutorials, try to set up the UserPathFunction using the above equations of motion. Most potential errors encountered will be in setting up the dynamics and their derivatives as they are quite intricate. Be sure to go over your derivations and inputs to the source code to make sure the equations are correct.

Main Function

In the main() function, included in 5-DROtoL2.cpp, enter the name of the problem, e.g. DROtoL2:

```
Driver* driver = new Driver("DROtoL2");
```

Building and Running the Problem

Compile and run the fourth tutorial the same way you did for the previous tutorials.

Hohmann Transfer with GMAT Utilities

Contents

Problem Formulation 58
Driver 60
UserPointFunction 62
UserPathFunction 63
Main Function 63
Using GMAT Utilities in the PostProcessing Method 64
Extract the Optimized Data 65
Setup a GMAT Solar System Dynamical Model 65
Setup a Spice Ephemeris and Create an EphemerisSmoother Object 66
Convert Between Reference Frames, State Representations, and Times 67
Earth-Centered Cartesian to Keplerian Transformation 67
Express Solution in the Sun-Centered Ecliptic Frame 68
Calculate the Distance from the Moon 69
Building and Running the Problem 70

Problem Formulation

Minimize the total ΔV of an impulsive Hohmann transfer from a smaller circular orbit to a larger circular orbit with free final time. All quantities are nondimensional.

$$Min\ J = \Delta V_{u1} + \Delta V_{v1} + \Delta V_{u2} + \Delta V_{v2}$$

Subject to:

$$\dot{r} = u$$

$$\dot{\theta} = \frac{v}{r}$$

$$\dot{u} = \frac{v^2}{r} - \frac{1}{r^2}$$

$$\dot{v} = -\frac{uv}{r}$$

Initial conditions:

$$t_0 = 0$$

$$r(t_0) = 1.05$$

$$\theta(t_0) = 0$$

$$u(t_0) = 0 - \Delta V_{u1}$$

$$v(t_0) = \frac{1}{\sqrt{1.05}} - \Delta V_{v1}$$

Final conditions:

$$r(t_f) = 1.5$$

$$u(t_f) = 0 + \Delta V_{u2}$$

$$v(t_f) = \frac{1}{\sqrt{1.5}} + \Delta V_{u2}$$

Variable bounds:

$$0 < t \leq 6\pi$$

$$0 < r(t) \leq 10$$

$$0 \leq \theta(t) \leq 12\pi$$

$$-10 \leq u(t) \leq 10$$

$$-10 \leq v(t) \leq 10$$

$$-0.1 \leq \Delta V_{u1}, \Delta V_{v1}, \Delta V_{u2}, \Delta V_{v2} \leq 0.1$$

States: r, θ, u, v

Controls: N/A

Static Parameters: $\Delta V_{u1}, \Delta V_{v1}, \Delta V_{u2}, \Delta V_{v2}$

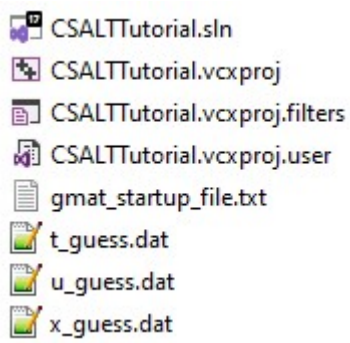
The equations of motion represent a planar system in a rotating frame, with the components of velocity along the radial, and tangential directions (see tutorial 3 for visualization). Tutorial 3 used low thrust with two control variables, however this tutorial optimizes for an impulsive Hohmann transfer from an inner circular orbit to an outer circular orbit. Since impulsive ΔV's are instantaneous and not continuous like finite burns or low thrust, they are not represented as control variables in the direct transcription. However we can model impulsive maneuvers in CSALT using an additional capability called static parameters. Static parameters are values in either the dynamics or the constraints that may be varied by the optimizer to achieve a local extremum. Static parameters are invoked and used much like states or controls in the various CSALT tutorial classes and methods.

Once the Hohmann transfer is solved, we will use some basic GMAT utilities to manipulate the optimized trajectory in the PostProcess() method of the Driver class. The following will be done with GMAT utilities on the trajectory:

- Define solar system bodies, reference frames, and perturbing bodies, in this case the sun and the moon (Luna)

- Set up an ephemeris object that uses Spice ephemeris data to provide real positions and velocities of the solar system bodies.
    - The ephemeris object uses Spice ephemeris data to construct polynomials that approximate the position and velocity values of selected solar system bodies over a given timespan. This technique typically speeds up computations and ensures smooth derivatives.
- Convert between reference frames, state representations, and times
    - Convert the optimized Hohmann transfer from cartesian to Keplerian elements
    - Example of converting UTC time instances to the A1 time system
    - Convert the optimized cartesian Earth-centered equatorial frame to a sun-centered ecliptic reference frame
    - Print the distance from the center of the moon to the vehicle

Before we can use GMAT capabilities, we will need to have the gmat_startup_file.txt in the same folder where your CSALTTutorial Visual Studio Project files are located:



Driver

In the driver source code, we will need to use some specific header files from the GMAT Base library:

```
#include "StateConversionUtil.hpp"
#include "CoordinateConverter.hpp"
#include "EphemSmoother.hpp"
#include "TransformUtil.hpp"
```

We will use a mesh interval spanning 11 equidistant points between [-1, 1] with 8 mesh points between each interval. The Static Parameters are set much like the state or control parameters with an integer declaring the number of static parameters, the upper and lower bounds, and a starting guess. Below we give a maximum nondimensional magnitude for the static parameters (ΔV's) a value of 0.1 and set that as the lower and upper bounds in staticUpperBound() and staticLowerBound(). We also set the number of static variables and give an initial guess of zero for each ΔV.

```
void Driver::SetPointPathAndProperties()
{
   pathObject = new PathObj();
   pointObject = new PointObj();

   // Set CSALT and SNOPT-related parameters in this method
   // See CsaltDriver.hpp for some of the available options
   maxMeshRefinementCount = 25;
   feasibilityTolerances(0) = 1.0e-6;
   optimizationMode[0] = "Minimize";
}

void Driver::SetupPhases()
{
    //Set mesh properties
    RadauPhase* phase1 = new RadauPhase();
    // Initial guess modes: "LinearNoControl", "LinearUnityControl",
    // "LinearCoast", "OCHFile", "GuessArrays"
    std::string initialGuessMode = "LinearNoControl";
    Rvector meshIntervalFractions(11, -1.0, -0.8, -0.6, -0.4, -0.2, 0.0, 0.2, 0.4, 0.6, 0.8, 1.0);
    IntegerArray meshIntervalNumPoints = { 8, 8, 8, 8, 8, 8, 8, 8, 8, 8 };

    //Set time properties
    Real timeLowerBound = 0.0;
    Real timeUpperBound = 6.0 * GmatMathConstants::PI;
    Real initialGuessTime = 0.0;
    Real finalGuessTime = 2.0 * GmatMathConstants::PI;

    //Set state properties
    Integer numStateVars = 4;
    Rvector stateLowerBound(4, 0.0, 0.0, -10.0, -10.0);
    Rvector initialGuessState(4, 1.0, 0.0, 0.0, 1.0);
    Rvector finalGuessState(4, 1.1, 1.0 * GmatMathConstants::PI, 0.1, 1.0 / sqrt(1.1));
    Rvector stateUpperBound(4, 10.0, 12.0 * GmatMathConstants::PI, 10.0, 10.0);

    // Set static parameter properties
    Integer numStaticVars = 4;
    Real dvMagBD = 0.1;
    Rvector staticUpperBound(4, dvMagBD, dvMagBD, dvMagBD, dvMagBD);
    Rvector staticLowerBound(4, -dvMagBD, -dvMagBD, -dvMagBD, -dvMagBD);
    Rvector staticGuess(4, 0.00, 0.00, 0.00, 0.00);
```

With the variables for the static parameters defined, we then apply them to the phase object with the appropriate setter methods. Notice since we do not have any control variables, those are missing from the phase object.

```
// Set mesh properties
phase1->SetInitialGuessMode(initialGuessMode);
phase1->SetNumStateVars(numStateVars);
phase1->SetNumStaticVars(numStaticVars);
phase1->SetMeshIntervalFractions(meshIntervalFractions);
phase1->SetMeshIntervalNumPoints(meshIntervalNumPoints);
phase1->SetStateLowerBound(stateLowerBound);
phase1->SetStateUpperBound(stateUpperBound);
phase1->SetStateInitialGuess(initialGuessState);
phase1->SetStateFinalGuess(finalGuessState);
phase1->SetTimeLowerBound(timeLowerBound);
phase1->SetTimeUpperBound(timeUpperBound);
phase1->SetTimeInitialGuess(initialGuessTime);
phase1->SetTimeFinalGuess(finalGuessTime);

phase1->SetNumStaticVars(numStaticVars);
phase1->SetStaticLowerBound(staticLowerBound);
phase1->SetStaticUpperBound(staticUpperBound);
phase1->SetStaticGuess(staticGuess);

phaseList.push_back(phase1);
```

UserPointFunction

The EvaluateFunctions() method in the UserPointFunction class is where we will apply the ΔV constraints to the problem. In the EvaluateFunction() method below, we extract the static parameters with the DVs = GetStaticVector(p) method, where p is the phase number. Our static parameter vector has 4 elements with the first two being the components of the first ΔV (dv1) and the last two elements being the components of the second ΔV (dv2). We also extract the necessary times and states to set up our constraints. The cost function is the minimum of the sum of the magnitudes of both dv1 and dv2. We calculate those with the Norm() method available to the Rvector data type. Next the constraints are formed with the states and the static parameters. Finally we set the bounds on the constraints, which are all of the equality type.

```
void PointObj::EvaluateFunctions()
{
   // Extract parameter data

   Rvector DVs = GetStaticVector(0);
   Rvector dv1(2, DVs(0), DVs(1));
   Rvector dv2(2, DVs(2), DVs(3));


   Rvector initStateVec = GetInitialStateVector(0);
   Rvector finalStateVec = GetFinalStateVector(0);

   Real initTime = GetInitialTime(0);

   Real r0 = initStateVec(0);
   Real th0 = initStateVec(1);
   Real u0 = initStateVec(2);
   Real v0 = initStateVec(3);

   Real rf = finalStateVec(0);
   Real uf = finalStateVec(2);
   Real vf = finalStateVec(3);


   Real dv1Mag = dv1.Norm(), dv2Mag = dv2.Norm();
   Rvector minDV(1, dv1Mag + dv2Mag);
   SetFunctions(COST, minDV);

   Rvector algFunctions(8, initTime, r0, th0, u0 + dv1(0), v0 + dv1(1),
       rf, uf + dv2(0), vf + dv2(1));

   Real r0value = 1.05;
   Real rfvalue = 1.5;
   Rvector lowerBound(8, 0.0, r0value, 0.0, 0.0, 1.0 / sqrt(r0value), rfvalue, 0.0, 1.0 / sqrt(rfvalue));
   Rvector upperBound(8, 0.0, r0value, 0.0, 0.0, 1.0 / sqrt(r0value), rfvalue, 0.0, 1.0 / sqrt(rfvalue));

   SetFunctions(ALGEBRAIC, algFunctions);
   SetFunctionBounds(ALGEBRAIC, LOWER, lowerBound);
   SetFunctionBounds(ALGEBRAIC, UPPER, upperBound);
```

UserPathFunction

The setup UserPathFunction class is very similar to the UserPathFunction setup of tutorial 3, except we don't have any controls or thrust in the state dynamics. The associated control Jacobian is an empty 4 x 0 matrix.

Main Function

The source file with the main() function will need some additional definitions in order to use the GMAT capabilities. First, we need to include the Moderater.hpp headerfile and create a global pointer of the Moderator class type:

```
#include "Driver.hpp"
#include "Moderator.hpp"

static Moderator* mod = NULL;
```

Then we create a Moderator instance, and check if it can initialize with the gmat_startup_file.txt, which is required to use GMAT utilities. The gmat_startup_file.txt needs to be located in the same folder as the

tutorial project. If the file could be found and is valid, we run the CreateDefaultParameters() method from the Moderator. Otherwise, an error is thrown:

```
// Simple program main() function
int main() {
    // Prompt user to hit a key to run
    std::cout << "Press Enter to Run";
    std::cin.ignore();

    std::string startUpFile = "gmat_startup_file.txt";

    mod = Moderator::Instance();
    if (mod == NULL || !(mod->Initialize(startUpFile))) {
        std::cout << "Moderator failed to initialize! Unable to run CSALT with GMAT." << std::endl;
        exit(EXIT_FAILURE);
    }
    mod->CreateDefaultParameters();
```

Next, give the Driver input string an appropriate name:

```
Driver* driver = new Driver("HohmannGMAT");
```

Finally, assuming the rest of the program ran correctly, we deallocate any overhead memory that was used with the Moderator just before the return statement with

`Moderator::Instance()->Finalize()`:

```
    // Run the driver
    Integer exitFlag = driver->Run();

    // Prompt user to hit a key to exit
    std::cout << "\nPress Enter to Exit";
    std::cin.ignore();

    // Delete driver instantiation
    delete driver;

    Moderator::Instance()->Finalize();

    // return exit flag to caller
    return exitFlag;
}
```

Using GMAT Utilities in the PostProcessing Method

In the PostProcessing() method of the CsaltDriver class, we perform some manipulations of the optimized trajectory to showcase some of the capabilities of GMAT within CSALT. Note that these GMAT capabilities can also be used in the Point and Path object classes by adding the desired GMAT objects as class variables and setting these variables with user-defined setter/getter functions in the driver.

Extract the Optimized Data

After optimization, the CsaltDriver parent class exposes the Trajectory object to the PostProcessing() method, and we can extract any of the times, states, controls, and static parameters. The number of integration steps is found with the GetNumRows() method. Next, we convert the time, states, and ΔVs into dimensional units with the length dimension being the Earth's equatorial radius, and the time dimension being the inverse of the mean motion of a circular orbit with a radius equal to that of Earth. At the end of this section, we print out the minimized ΔV in km/s.

```
void Driver::PostProcess(Trajectory* traj) {
    Phase* phase = traj->GetPhaseList()[0];
    Rvector time = phase->GetTimeVector();
    DecisionVector* dv = phase->GetDecisionVector();
    Rmatrix states = dv->GetStateArray();
    Rvector staticParams = dv->GetStaticVector();

    Integer n = states.GetNumRows();

    // Convert initial guess state to Keplerian orbital elements
    Real earthMu = 398600.4418;
    Real earthRadius = 6378.137;
    Real lstar = earthRadius;
    Real tstar = sqrt(pow(lstar, 3.0) / earthMu);

    Rvector t = time * tstar;
    Rvector r = states.GetColumn(0)*lstar;
    Rvector th = states.GetColumn(1);
    Rvector u = states.GetColumn(2)*lstar/tstar;
    Rvector v = states.GetColumn(3)*lstar/tstar;
    Rvector dv1(2, staticParams(0), staticParams(1));
    dv1 *= lstar / tstar;
    Rvector dv2(2, staticParams(2), staticParams(3));
    dv2 *= lstar / tstar;

    std::cout << "Minimum DV = " << dv1.Norm() + dv2.Norm() << " km/s" << std::endl;
```

Setup a GMAT Solar System Dynamical Model

We setup up a solar system dynamical model using GMAT and the JPL/NAIF cspice library. In this case, we will use the default de405.bsp binary ephemerides. First, we instantiate a SolarSystem object, and three CelestialBody objects for the Earth, Luna, and Sun:

```
SolarSystem* solarSys = new SolarSystem("HohmannTransferSolarSys");

// Declare and instantiate Earth CelestialBody object
CelestialBody* earth; // object to store Earth body
earth = solarSys->GetBody("Earth");
// Declare and instantiate Sun CelestialBody object
CelestialBody* sun;
sun = solarSys->GetBody("Sun");
// Declare and instantiate Luna CelestialBody object
CelestialBody* luna;
luna = solarSys->GetBody("Luna");
```

Next, we instantiate three reference frames that we will use to convert between different states with CoordinateSystem objects. These frames are the Modified J2000 Equatorial frame centered at the Earth, a Modified J2000 Ecliptic frame centered at the Sun, and a Modified J2000 Ecliptic frame centered at Luna. The objects are declared with the CoordinateSystem::CreateLocalCoordinateSystem() method.

```
// Declare and instantiate reference frames
CoordinateSystem* earthMJ2000Eq;
earthMJ2000Eq = CoordinateSystem::CreateLocalCoordinateSystem(
    "EarthMJ2000Eq", "MJ2000Eq", earth, NULL, NULL, earth->GetJ2000Body(), solarSys);
CoordinateSystem* sunMJ2000Ec;
sunMJ2000Ec = CoordinateSystem::CreateLocalCoordinateSystem(
    "SunMJ2000Ec", "MJ2000Ec", sun, NULL, NULL, sun->GetJ2000Body(), solarSys);
CoordinateSystem* lunaMJ2000Ec;
lunaMJ2000Ec = CoordinateSystem::CreateLocalCoordinateSystem(
    "LunaMJ2000Eq", "MJ2000Eq", luna, NULL, NULL, luna->GetJ2000Body(), solarSys);
```

Setup a Spice Ephemeris and Create an EphemerisSmoother Object

Here we create an ephemeris smoother which we will use to obtain the ephemerides of Luna and the Sun at given times. The “smoother” is used so that both computational time is sped up, and smooth derivatives of the ephemerides is ensured.

First, we create a vector of CelestialBody objects in which we will store all the third bodies for use in the ephemeris smoother.

```
StringArray pertBodies;
pertBodies.push_back("Sun");
pertBodies.push_back("Luna");

// Get NAIF IDs associated with each body
std::vector<CelestialBody*> thirdBodyObjs;
for (Integer ii = 0; ii < pertBodies.size(); ii++)
    thirdBodyObjs.push_back(solarSys->GetBody(pertBodies.at(ii)));
```

Next, we instantiate the EphemSmoother object while defining the epoch window from which we will draw states from the cspice binary file. The epoch window is defined in terms of Modified Julian Date.

Within the EphemSmoother object, we create a smoothed ephemeris for both Luna and the Sun with the CreateSmoothedEphemForCelestialBody() method.

```
 // Ephemeris smoother class used to extract ephemerides more quickly and accurately
EphemSmoother* ephemerisSmoother;

// Define EphemSmoother settings
Integer ephemSmoother_numRegionsPerRev = 359;
Integer ephemSmoother_numRegions = 359;
Real ephemSmoother_H = 1.0;
std::string ephemSmoother_StepSizeType = "h";
Real ephemSmoother_splineT0 = 25198.0;
Real ephemSmoother_splineTf = 40000.0;

// Instantiate and initialize EphemSmoother object using transfer and science orbit body list.
ephemerisSmoother = new EphemSmoother(ephemSmoother_numRegionsPerRev, ephemSmoother_numRegions,
    ephemSmoother_H, ephemSmoother_StepSizeType, ephemSmoother_splineT0, ephemSmoother_splineTf);
for (Integer ii = 0; ii < thirdBodyObjs.size(); ii++)
{
    ephemerisSmoother->CreateSmoothedEphemForCelestialBody(earth, thirdBodyObjs.at(ii),
        earthMJ2000Eq, ephemSmoother_splineT0, ephemSmoother_splineTf, ephemSmoother_numRegionsPerRev);
}
```

Convert Between Reference Frames, State Representations, and Times

In this section, we will convert the optimized Hohmann state history into different coordinate frames and types. The following transformations are performed:

- Convert from cartesian states to Keplerian states
- Convert from Earth J2000 to sun-centered ecliptic frame
- Calculate the distance from Luna to the spacecraft along the entire trajectory

Earth-Centered Cartesian to Keplerian Transformation

The optimized states are not yet in an inertial cartesian representation, but rather in a frame rotating with the vehicle. Assuming no out-of-plane motion (No movement in +/- Z direction), the transformation from the rotating frame to inertial cartesian is:

$$X = r\cos\theta$$

$$Y = r\sin\theta$$

$$V_X = u\cos\theta - v\sin\theta$$

$$V_Y = u\sin\theta + v\cos\theta$$

In the following code snippet, we both convert the optimized state history into ECI and then express them in Keplerian elements. We first instantiate a StateConversionUtil object and then use its Convert() method to convert from ECI to Keplerian. Note that the Convert() method requires the Rvector6 data type, and will not accept the more general Rvector or Rmatrix data types. This means we have to copy the state history for each timestep into a Rvector6 array and call then call the Convert() method. After the state conversion, the Keplerian elements are written to a file, statesKep.dat.

```
// Get key Earth parameters
Real mu, flattening, eqRadius;
mu = earth->GetGravitationalConstant();
flattening = earth->GetFlattening();
eqRadius = earth->GetEquatorialRadius();
 // Convert phase state to Keplerian state representation
StateConversionUtil* stateConvUtil;
Rvector6 stateKep;
Rvector6 stateCart;
Rmatrix statesCart(n, 6);
Rmatrix statesKep(n, 6);
for (Integer i = 0; i < n; ++i) {
    statesCart(i, 0) = stateCart(0) = r(i) * cos(th(i));
    statesCart(i, 1) = stateCart(1) = r(i) * sin(th(i));
    statesCart(i, 3) = stateCart(3) = u(i) * cos(th(i)) - v(i) * sin(th(i));
    statesCart(i, 4) = stateCart(4) = u(i) * sin(th(i)) + v(i) * cos(th(i));
    stateKep = stateConvUtil->Convert(stateCart, "Cartesian", "Keplerian", mu, flattening, eqRadius);
    for (Integer j = 0; j < 6; ++j)
        statesKep(i, j) = stateKep(j);
}

FILE* fid;
fid = fopen("statesKep.dat", "w");
for (Integer i = 0; i < n; ++i)
    fprintf(fid, "% .16f % .16f % .16f % .16f % .16f % .16f % .16f\n",
        t(i),
        statesKep(i, 0),
        statesKep(i, 1),
        statesKep(i, 2),
        statesKep(i, 3),
        statesKep(i, 4),
        statesKep(i, 5));
fclose(fid);
```

Express Solution in the Sun-Centered Ecliptic Frame

Now we will express the optimized state history in a frame centered at the Sun. The conversion is done in a loop like the Keplerian conversion above, but now we also need to convert time from UTC to A1 time as that is what the conversion routine requires. We instantiate a timeConverter object and use it inside the loop with the optimized time array. Note that we must convert our optimized time in seconds into Modified Julian years, and that we have selected the beginning UTC time arbitrarily as 32198.0 MJD. The Sun-centered ecliptic states are written to the file CartStates_sunMJ2000Ec.dat. Note that the value 3.1688087814029e-8 must be multiplied by the trajectory time t, since utcTime is in MJD, not seconds.

```
TimeSystemConverter* timeConverter;
timeConverter = timeConverter->Instance();

Integer a1TimeID = timeConverter->GetTimeTypeID("A1Mjd");
Integer utcTimeID = timeConverter->GetTimeTypeID("UtcMjd");
Real utcTime, a1Time;
utcTime = 32198.0; // should be within T0 anf Tf bounds set for the smoothed ephemeris spline

// Convert phase state to Keplerian coordinates in the Sun-centered ecliptic frame
Rmatrix CartStates_sunMJ2000Ec(n, 6);
Rvector6 CartState_sunMJ2000Ec;
for (Integer i = 0; i < n; ++i) {
    a1Time = timeConverter->Convert(utcTime + 3.1688087814029E-08 * t(i), utcTimeID, a1TimeID);
    for (Integer j = 0; j < 6; ++j)
        stateCart(j) = statesCart(i, j);
    CartState_sunMJ2000Ec = TransformUtil::TransformOrbitalState(stateCart,
        a1Time, earthMJ2000Eq, "Cartesian", sunMJ2000Ec, "Cartesian", "Spline", ephemerisSmoother);
    for (Integer j = 0; j < 6; ++j)
        CartStates_sunMJ2000Ec(i, j) = CartState_sunMJ2000Ec(j);
}

fid = fopen("CartStates_sunMJ2000Ec.dat", "w");
for (Integer i = 0; i < n; ++i)
    fprintf(fid, "% .16f % .16f % .16f % .16f % .16f % .16f % .16f\n",
        t(i),
        CartStates_sunMJ2000Ec(i, 0),
        CartStates_sunMJ2000Ec(i, 1),
        CartStates_sunMJ2000Ec(i, 2),
        CartStates_sunMJ2000Ec(i, 3),
        CartStates_sunMJ2000Ec(i, 4),
        CartStates_sunMJ2000Ec(i, 5));
fclose(fid);
```

Calculate the Distance from the Moon

In this last exercise we will obtain the distance from the optimized trajectory to the center of Luna. This is done in a loop like the previous sections. First we obtain Luna's cartesian position at a given time, then we use the Norm() method on that Rvector data type to get the total distance. After obtaining the distances, the initial and final distances to Luna are printed to the console.

```
// Get distance from Moon's center
Rvector6 KepState_lunaMJ2000Ec;
Rvector MoonPosition(3);
Rvector MoonDistance(n);
for (Integer i = 0; i < n; ++i) {
    a1Time = timeConverter->Convert(utcTime + 3.1688087814029E-08 * t(i), utcTimeID, a1TimeID);
    for (Integer j = 0; j < 6; ++j)
        stateCart(j) = statesCart(i, j);
    KepState_lunaMJ2000Ec = TransformUtil::TransformOrbitalState(stateCart,
        a1Time, earthMJ2000Eq, "Cartesian", lunaMJ2000Ec, "Cartesian", "Spline", ephemerisSmoother);
    for (Integer j = 0; j < 3; ++j)
        MoonPosition(j) = KepState_lunaMJ2000Ec(j);
    MoonDistance(i) = MoonPosition.Norm();
}
std::cout << "Initial distance from the Moon = " << MoonDistance(0) << " km" << std::endl;
std::cout << "Final distance from the Moon = " << MoonDistance(n - 1) << " km" << std::endl;

if (solarSys)
    delete solarSys;
if (ephemerisSmoother)
    delete ephemerisSmoother;
```

Building and Running the Problem

Compile and run the sixth tutorial the same way you did for the previous tutorials.